

MULTIDISCIPLINARY PROJECT FINAL REPORT

VinPRO

Vineyard Pruning with Robots through Collaborative Learning in Virtual Reality

Avantaggiato Vincenzo, Master of Computer Engineering, Politecnico di TO

(signature) Avantaggiato Vincenzo

Eusebio Alberto, Master of Computer Science and Engineering, Politecnico di MI

(signature) Alberto Eusebio

Ghianni Riccardo, Master of Quantum Engineering, Politecnico di TO

(signature) Ghianni Riccardo

Risso Francesco, Master of Computer Engineering, Politecnico di TO

(signature) Francesco

Tahirović Faik, Master of Automation and Control Engineering, Politecnico di MI

(signature) Faik Tahirović

Troilo Eleonora, Master of Mechanical Engineering, Politecnico di TO

(signature) Eleonora Troilo

Vallino Riccardo, Master of Aerospace Engineering, Politecnico di TO

(signature) Riccardo Vallino

Vignoli Lorenzo, Master of Mechanical Engineering, Politecnico di MI

(signature) Lorenzo Vignoli

Principal Academic Tutor:

Marcello Chiaberge, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

(signature)_____

Other Academic Tutors:

Luca Bascetta, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di MI

Mauro Martini, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

Marco Ambrosio, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

Alessandro Navone, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

Brenno Tuberga, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

Luigi Mazzara, Dipartimento di Elettronica e Telecomunicazioni, Politecnico di TO

External Tutor(s):

Marta Niccolini, YANMAR R&D Europe Srl

1 Executive Summary

Introduction

Today, vineyard pruning still heavily depends on the expertise of skilled human labor. However, an increasing number of research initiatives are working to bridge the gap between traditional manual pruning and fully autonomous robotic systems. VinPro, a project supported by the Alta Scuola Politecnica, is one such initiative paving the way forward.

Developed in collaboration with academic partners Pic4Ser, Politecnico di Torino (PoliTo), and Politecnico di Milano (PoliMi), alongside industrial partner Yanmar Europe R&D, the project seeks to deliver a fully integrated robotic pruning system composed of:

1. A custom-designed gripper tailored for vineyard environments,
2. An advanced computer vision system to detect sacrificial branches,
3. A control algorithm that manages and optimizes the cutting action,
4. A virtual reality (VR) platform that models the ideal vine and simulates pruning strategies.

At the core of the system is a Kinova Gen3 LITE robotic arm, mounted on a mobile rover developed by PoliTo's robotics laboratory, forming the physical backbone of the robotic pruning platform.

1. Gripper Design

To interface with the Kinova robotic hand without dismantling it, the team engineered a lightweight, 3D-printed PA6 polymer housing—a kind of mechanical “glove” that integrates the actuation mechanism for the pruning shear. Two drive solutions were evaluated:

- A pneumatic motor powered by onboard compressed air,
- A compact electric motor connected to a battery pack.

The electric motor was ultimately selected for its reduced footprint and seamless integration with lab resources. An Arduino microcontroller translates control commands from the central system, triggering the cutting action via digital signals.

2. Computer Vision

With the mechanical structure in place, the focus shifted to modeling the vine's anatomy. Using a dataset of various grapevine images, the team developed a deep learning-based pipeline to:

- Detect nodes along the vine,
- Classify branch types and node types
- Estimate spatial relationships among structural elements.

For each input image, a directed graph is produced as output whose nodes have their own category, while edges correspond to the branches. Every node has at most one parent.

3. Control and Learning

A pruning policy algorithm has been developed to automatically decide the cutting points of the plant in the image. This points are then converted into 3D coordinates and are used as inputs to the robotic arm controlling part.

A motion-planning algorithm calculates the path toward each pruning point, orienting the cutter precisely, avoiding collisions, before initiating the cut.

Cameras on the gripper feed real-time image data into the graph-generation engine, which identifies the necessary structures and passes them to the control module. The shear is then actuated using ROS 2 Control libraries, which interface with the Arduino board. Once a cut is confirmed, the system proceeds seamlessly to the next task.

4. Virtual Reality Simulation

To support human–machine collaboration, a virtual reality (VR) interface was developed. With detailed 3D models of vines—including trunks, branches, grapes, and leaves—the team built a virtual vineyard environment. This enabled the simulation of different growth scenarios, pruning strategies, and expected outcomes. The modular nature of the model allows for easy adaptation to different vineyard stages or training purposes.

Conclusions

Over two years of development, the VinPro project has successfully delivered a proof-of-concept system capable of:

- Modeling vine architecture through computer vision,
- Planning and executing cuts with a robotic arm and precision gripper,
- Learning and adapting through iterative control refinement,
- Simulating pruning operations in virtual environments.

This work represents a significant advancement towards semi-autonomous vineyard maintenance, with the long-term potential to rival human expertise in precision and adaptability.

2 Introduction

Wine production involves several key processes that directly affect both the quality of the final product and the productivity of the vineyards. Among these, pruning plays a central role due to the high level of precision it requires and the significant costs it entails. In the field, pruning is a repetitive operation and, with few exceptions, is still performed manually. This demands highly skilled labour and considerable investment of time. For fruit production, pruning alone can account for up to 25% of farmers' annual labour costs [1]. In vineyards, the task not only requires precision but also adherence to pruning techniques specific to each plant variety. This adds further complexity to the challenge of automating the process.

Over the years, different solutions have been explored. However, the high precision required often leads to complex systems, which often tend to vary greatly from one another. Some solutions proved effective in terms of accuracy but were too slow to be practically viable, while others operated faster but produced coarse and imprecise results, failing to meet the quality standards required by wine producers. In addition to technical challenges, vineyards owners face increasing labour shortages, which make manual pruning more difficult to sustain in the long term. Indeed, vineyards have to be pruned in specific period of the year to ensure the best harvest. This issue underlines the importance of automation as a tool not only to reduce costs, but also to improve sustainability, limiting the reliance on seasonal labour and reducing environmental footprint.

With the support of external stakeholders, the **VinPRO project** aimed at advancing vineyard pruning by integrating cutting-edge technologies with human expertise in an innovative way. The aim was to move beyond the coarse, indiscriminate approach of some automated solutions, while preserving the efficiency and precision of manual pruning, but reducing the cost and time involved. To achieve this, expert knowledge was incorporated into the system, ensuring that established pruning techniques could be preserved and transferred to the robot.

This role was taken by **Cantina 366** (Agl  ,TO), an Italian winemaker interested in the final success of the solution, which also provided facilities for field testing. The owner of Cantina 366 personally dedicated time to demonstrate different pruning techniques, highlighting how approaches vary depending on plant typology and explaining the advantages of each method. This contribution was essential for understanding how to prefer one pruning strategy to another and how to implement it in the algorithm of the robot. **YANMAR R&D** contributed with technological and commercial resources, helping to place the project within the broader landscape of agricultural machinery and ensuring its competitiveness. In recent years, this firm has invested a lot of resources for developing new agricultural technologies, under its programme 'YANMAR Smart Agriculture' [2]. Several projects aim to design unmanned and automatic system useful in all phases of the production. Another key partner was **PIC4SeR**, the Interdepartmental centre for Service Robotics at Politecnico di Torino and Politecnico di Milano [3], which supported the technological development and integration of robotic and sensor systems. In particular, PIC4SeR showed a strong interest in obtaining a working prototype able to reliably detect pruning points while complying with expert pruning rules. Similar projects are carried out in the centre, focusing on collaborative robotics and precision agriculture and letting human expertise and robotic autonomy converge.

In this context, the VinPRO team worked for a clear and specific goal: design and implement an innovative robotic system capable of pruning vineyards autonomously. To achieve this, following the guidance of the tutors, the work was organised into four interrelated **Work Packages** (WPs), each with its own tasks:

- **WP1:** Design of an automatic end-effector tailored for pruning and integrated into an existing robotic platform, provided by PIC4SeR. The focus was on ensuring that the tool could adapt to different branch sizes and hardness, while maintaining precision and robustness during the cutting operation. It is also essential to avoid any kind of damage to the vine structure.

- **WP2:** Development of a Computer Vision pipeline able to detect branch structure and identify optimal cutting points, through an iterative process to train and optimise of AI models for high precision of the robot's limited computational hardware. Particular attention was devoted to balancing accuracy with computational efficiency, so that detection could run in real time under field conditions. This module should provide reliable input for the control algorithm.
- **WP3:** Design of a control system for the robotic platform that takes the cutting point location as input and calculates the final position of the robot, avoiding any type of collision. This required advanced motion planning and continuous adjustment of trajectories to account for the complexity of vineyard environments, where irregular branch structures make manoeuvring particularly challenging.
- **WP4:** Creation of a Virtual Reality environment aimed at training the detection of the cutting points and allow operators to familiarise themselves with the collaborative human-robot pruning approach in a risk-free setting. Beyond training, this environment also offered a valuable testing ground to simulate interactions between robot and operator before deploying the system in real vineyards.

The results of these four WPs were strongly interconnected: the accuracy of WP2's vision system for example directly affected WP3's motion planning, while WP1's mechanical reliability determined the effectiveness of the final cut. Instead, WP4 relied on data from all other WPs to create a realistic training and simulation environment, ensuring that the system could be validated under controlled conditions.

The team worked to achieve high level of precision and accuracy in vine pruning, with the potential for continuous improvement even after the project's completion. The work progressed through four main phases. The **first phase** involved an analysis of stakeholders, needs and requirements of the solution and a review of the state of the art, including literature and commercial solutions, followed by organization within the team and cost planning. The **second phase** saw each WP working in parallel and independently on its own tasks. During this period, the team also carried out a field visit to Cantina 366 in Agliè to collect data on pruning techniques and field characteristics. In the **third phase**, the outputs of the different WPs were integrated and the overall system validated. The **fourth and final phase** consisted of a validation campaign to evaluate the performance of the complete system.

Throughout the project, every two weeks of work, the team scheduled a meeting with the tutors of PIC4SeR, in order to monitor the progress, align with stakeholder expectations, and timely adjust to the work plan when necessary. These meetings were mostly held online via Microsoft Teams, allowing each subgroup to present their advances and enabling collective decision-making on the subsequent steps. Tools such as GitHub were used for version control and code sharing, ensuring coordination and traceability across the WPs. At the end of the work, a final validation campaign was carried out in the laboratory of PIC4SeR, using a grapevine mock-up, in order to evaluate the integrated system under controlled conditions before the field deployment.

This report retraces the path followed by the VinPRO team during its work. After this introduction (Section 2), Section 3 presents the expectations identified through stakeholder analysis and the corresponding requirements for the final system. These requirements concern not only performance, but also environmental impact, field adaptability, and interaction with human experts or farmers. Section 4 provides the results of the state of the art review, focusing on the different systems involved in the project: autonomous robotic pruning techniques, the tree perception (branches, seeds, buds, ...), the estimation of the cutting point, based on the different pruning techniques, design of possible end-effectors and control techniques for the robotic arm. In this section, there are also some end-to-end solutions that have been implemented and an analysis of their strengths and their limitations. Section 5 describes the proposed solution and the contribution of each WP: detailing system specifications, concept selection, feasibility evaluation, and validation tests carried out on a grapevine mock-up. Finally, Section 6 summarises the project's main findings, offers recommendations for external institutions, and outlines possible future developments for the VinPRO project beyond the ASP cycle.

3 Users' Requirements

3.1 Stakeholders with Direct Interaction

This section identifies and describes the primary stakeholders who directly interact with the proposed pruning solution. Their needs, expectations, and operational constraints have been considered throughout the project to ensure that the system meets practical requirements and supports successful adoption in real-world scenarios. Additional stakeholders with indirect involvement will be addressed separately.

Pruning Supervisor

The pruning supervisor is responsible for continuously monitoring the operation, ensuring that the pruning robot remains in the correct operational state and that the pruning process is executed successfully. Their primary expectation is ease of supervision, requiring only relevant and concise operational information without being overloaded with unnecessary data. Additionally, they expect the robot to demonstrate high operational efficiency, minimal occurrence of fault states, and low frequency of mechanical or software failures, as such issues would significantly increase their workload and hinder operational continuity.

Robot Assembly Companies

Robot assembly companies, such as YANMAR, are responsible for manufacturing the pruning robots. Their primary expectation is that the assembly process can be readily automated, thereby reducing production complexity and improving the workload-to-profit ratio. They also require that all components be readily available on the market. While the overall system is tailored specifically for the pruning task, its individual components should remain standardized rather than custom-made, ensuring ease of procurement, scalability, and cost efficiency.

Robot Maintenance Personnel

Robot maintenance personnel are responsible for ensuring the continued operational reliability of the pruning robots. They expect the system to be modular, with components that are easily accessible and replaceable, similar to the requirements of the robot assembly companies. In addition, they require clear and comprehensive maintenance procedures, including detailed troubleshooting guidelines, to streamline the repair process, reduce downtime, and improve overall maintenance efficiency.

Large Vineyard Owners

Large vineyard owners expect the pruning solution to reduce both the cost of the pruning process and the associated labor requirements. Cost reduction is also supported by designing a robot with a low failure rate and minimal grape wastage. Additionally, they require the robot to operate reliably across diverse environments, including varying weather conditions and terrain types, enabling deployment in multiple vineyard settings.

During an interview with Francesco, one of the owners of Cantina366, he emphasized that their primary concern is keeping production costs low, noting that “a bottle costs around €4/L and must cover all expenses.” He also explained that, as a small vineyard, they have never attempted to implement an automated pruning solution, as the investment required would be excessive for their scale of operations. This further underscores the importance of minimizing the system’s initial acquisition cost to encourage broader adoption.

3.2 Stakeholders Benefiting from the Solution

Besides the stakeholders who directly interact with the solution, there are other groups that can benefit from its implementation, including consumers of wine, retail stores, hardware and service supplier companies, and research institutions. While these stakeholders may not be directly involved in operating or maintaining the pruning robot, they can still experience positive impacts resulting from its deployment.

Consumers of Wine

Wine consumers expect to maintain the same quality standards while benefiting from lower prices. The proposed solution can contribute to this outcome by increasing production efficiency and reducing overall production costs, which can directly influence the retail price of wine. Furthermore, by enabling precise and consistent pruning, the system can help ensure that harvested grapes meet specific quality requirements, supporting the production of wine that adheres to established quality standards.

Retail Stores

Retail stores can also benefit from the implementation of the solution, as reduced production costs can directly lower the wholesale price at which they purchase wine. In many cases, this cost reduction is even more immediate for retailers than for consumers. While a lower wholesale price may lead to a corresponding reduction in retail prices, the decrease in production costs could be significant enough to allow for improved profit margins. Additionally, increased production capacity enabled by the solution would make it easier for retail stores to maintain stock levels, ensuring a more consistent supply of products to meet consumer demand.

Hardware and Software Supplier Companies

Hardware and software supplier companies share a key expectation with robot assembly companies: that the production of their components can be easily automated. For these suppliers, automation not only reduces production complexity but also increases manufacturing throughput, enabling them to meet higher demand more efficiently. In addition, they expect component designs to follow established design patterns and industry standards, as this facilitates automation, simplifies the production process, and reduces the likelihood of manufacturing errors.

Research Institutions

Research institutions expect to gain valuable insights and, more importantly, access to extensive data related to vineyard operations. Such data would not only enable detailed analysis of the robot's performance but also support the design of vineyards in ways that optimize pruning efficiency. Furthermore, the robot's monitoring capabilities provide opportunities to study vineyard behaviour over time, allowing for enhanced production analysis and the identification of best practices in vineyard management. By facilitating continuous data collection, the solution can significantly advance research in both agricultural automation and viticulture.

3.3 Stakeholders Affected by the Solution

The final group of stakeholders includes those who may be indirectly impacted by the implementation of the pruning solution. This category comprises small vineyard owners, vineyard workers, and neighbouring communities.

Small Vineyard Owners

Small vineyard owners typically target buyers who prefer products with a more exclusive, handmade, and traditional character. Unlike large vineyard owners, they are less likely to benefit from the adoption of the solution, as noted by Francesco from Cantina366 during our discussions. Their main expectation is that market segmentation will remain unchanged, allowing them to retain their established customer base. They are concerned that the introduction of large-scale automated pruning solutions could enable larger producers to attract customers who currently value the artisanal qualities of small-scale

production. As such, they expect the solution to avoid creating market conditions that would displace their niche within the wine industry.

Workers

Vineyard workers share a common concern observed in many industries undergoing automation—the potential loss of employment. Over years of experience, these workers have developed valuable knowledge and expertise in pruning, which they rely on to earn a living. Their primary expectation is to retain their employment, even if the nature of their work changes. While they acknowledge that their expertise may be applied in a different capacity, they expect to play an active role in training the system, ensuring that the pruning process reflects established best practices and accumulated field knowledge.

Neighbouring Communities

Neighbouring communities expect the implementation of the pruning solution to maintain, and ideally enhance, their quality of life. This expectation can be met through several channels. One is the preservation of healthy employment levels, as an expansion in vineyard operations could create job opportunities for individuals with a range of educational backgrounds. Another is the safeguarding of the vineyards' touristic appeal, ensuring that automation does not diminish the cultural and economic benefits tourism brings to the local area. Finally, communities expect the solution to operate without generating negative impacts such as excessive noise, instability in the electrical grid, or environmental pollution, thereby preserving the overall well-being of the region.

3.4 Needs and requirements

This section identifies the main stakeholder needs and translates them into precise system requirements, defining how user expectations and operational constraints are directly transformed into measurable specifications. The requirements are then classified according to their nature and broader impact.

Through extensive stakeholder engagement, including interviews, surveys, and collaborative workshops, nine primary needs have been analysed to understand their underlying motivations, stakeholder priorities, and technical implications. Each need represents a critical aspect of the pruning operation, crucial to ensure successful system adoption.

Requirements are therefore clustered into five distinct categories: technical requirements that define system functionality and performance parameters; business requirements that address economical and operational concerns; individual impact requirements that consider effects on personal stakeholders; societal impact requirements that evaluate broader community implications; and regulatory ones that ensure compliance with existing standards or legislation.

Easier management of workload and workforce

The challenge of managing workload and workforce efficiently represents one of the most pressing concerns for pruning supervisors operating within modern vineyards. This need arises from the complexity of coordinating seasonal agricultural activities, where substantial pruning volumes must be accomplished within very limited timespans. This coordination is constrained by both high-quality standards, as pruning precision directly impacts vine health and subsequent grape yields, and worker safety. Moreover, traditional workforce coordination in vineyard pruning presents numerous obstacles, including variable skill level and physical capabilities, thus highlighting the need for an automated solution to mitigate this issue.

The cyclic characteristics of pruning activities mean that supervisors frequently collaborate with temporary or seasonal workers who may lack familiarity with specific vineyard configurations or established safety procedures. This situation generates continuous and not always satisfied demands for highly developed skills, or training, that impose considerable strain on supervisory resources and attention. Environmental conditions introduce another critical dimension to workforce management challenges: manual pruning operations demonstrate high sensitivity to weather variables, like precipitation or extreme temperatures potentially diminishing productivity.

To address these workforce management needs, the system must provide precise spatial awareness capabilities, being therefore able to measure its position and enabling supervisors to maintain real-time knowledge of robotic unit locations across vineyard operations. This capability ensures that the machine remains within designated operational zones and follows planned work sequences without requiring constant visual supervision. This technical requirement involves implementing GPS positioning systems and location tracking algorithms to make the vineyard operations autonomous.

The transformation from human workforce management to robotic oversight necessitates accessible system operations that accommodate varying levels of technical expertise. Therefore, a business-oriented requirement is that a new supervisor should be proficient with the system within three months, establishing reasonable learning expectations to avoid barriers to system implementation.

Comprehensive information access is also needed to surpass the limitations of traditional human oversight. The requirement, both technical and business-oriented, is that supervisors must be able to have full access to system information, providing them with detailed operational insights that were previously unavailable. The complete information access includes real-time operational status, diagnostic data, and performance analytics.

Regulatory issues

Regulatory compliance represents a critical concern spanning multiple stakeholders, each defending different perspectives on safety, environment, and operational standards. Pruning supervisors face direct liability concerns regarding worker security, while vineyard owners must ensure the system meets all applicable standards for business viability. Regulatory institutions establish and enforce these standards to protect workers, consumers, and the environment from potential harm associated with agricultural automation. Traditional farming regulations must be expanded as modern vineyard operations require additional standards when integrating robotic technology, as the intersection of automation with agriculture introduces complex compliance challenges.

Occupational safety represents the most immediate regulatory concern. To protect workers from hearing damage during robotic operations, the regulatory requirement is that noise emissions should not exceed 100dB for 2 hours per day, based on OSHA standards [33].

Worker protection leads to a further requirement, both regulatory and based on the impact on individual safety, that the system should present no threat to the security of workers and therefore incorporate safety mechanisms for workers who may interact with the VinPRO system.

A regulatory and technical specification is that the system should remain within the boundaries of its operating area, while as a regulatory mandate the system should not affect the safety of food products throughout the automated pruning process.

High workload in a short time frame

As mentioned earlier, the seasonal nature of vineyard pruning creates an intense concentration of labour, presenting significant challenges for multiple stakeholder groups. Pruning supervisors must coordinate massive workloads when weather conditions permit effective pruning; vineyard owners face the economic pressure of completing essential pruning tasks within optimal timing windows while ensuring vine health and grape quality; workers endure physically demanding schedules that may compromise both work quality and personal well-being. Traditional manual pruning methods struggle to meet these concentrated demands, often requiring extended working hours and additional labour, as synchronization with specific physiological requirements of vine growth cycles is fundamental.

The first requirement is that the system should respect the Estimated Time of Completion of the workload, and constitutes both a business and technical specification that ensures robotic operations match the productivity rates necessary for successful seasonal completion. This temporal precision eliminates the uncertainty associated with human labour availability and performance variability.

The system is required, as a technical concern, to perform at night, in presence of rain and wind to enable continuous operation beyond the environmental limitations that constrain human workers. The productive hours are thus maximized to reduce the risk of incomplete tasks due to adverse conditions.

Energy management is another key issue. The technical requirement that the charge/operation time ratio should be lower than 1 ensures that the robotic system spends more time working than recharging, preventing operational bottleneck during peak seasonal demands.

Robot reliability

System reliability is a fundamental need in autonomous operations, with differentiated stakeholders. Pruning supervisors require dependable equipment that minimizes unexpected failures during seasonal operations, while robot maintainers need systems designed for predictable interventions and clear diagnostic algorithms. Large vineyard owners need robots that justify significant capital investments, while reliable performance data are needed by robot assembler companies, hardware and services supply companies, and research institutions to support product development and market credibility. On the other hand, reliability in vineyard operations should extend beyond simple mechanical durability to gain widespread adoption, as exposure to dust, moisture, temperature variations, and various physical obstacle may compromise system integrity.

Critical system failures should be minimized, thus the requirement that the system unrecoverable breakdowns should not happen more than once per pruning season. This is both a technical and business specification to avoid compromising the entire pruning cycle – and the related investments.

Performance is another key aspect in reliability, leading to the technical specification that errors in pruning should happen less than 5% of the time. This accuracy requirement safeguards both vine health and grape production.

In order to ensure effective error and communication management, another key requirement is that managers and maintainers should be informed of errors. This is both a technical and business specification that ensures transparent operational monitoring and prompts corrective actions.

Pruning expertise

The preservation and application of accumulated pruning knowledge represents a critical challenge to ensure a smooth transition from manual methods to automated ones. Winemaking consultants and educators benefit from decades of specialized knowledge about optimal techniques, and to avoid impacting vine health and quality, robotic systems must replicate this expertise. Workers, in the meantime, bring valuable field knowledge and practical insights that must be integrated. As a consequence, the final product needs to preserve the nuanced understanding developed through generations while exploiting the opportunities offered by new technologies.

Automated branch recognition capabilities form the foundation of effective robotic pruning systems. The requirement that the system should automatically recognize branches that require pruning with an accuracy of 95% represents a technical specification that ensures robotic systems can replicate expert human decision-making in identifying optimal pruning locations.

Furthermore, the robot should improve performance through operational experience and expert input. Ideally, a technical requirement is that the accuracy of the prediction should increase with experience, a specification that can only be satisfied incorporating machine learning principles into agricultural automation. This adaptive capability both ensures increased effectiveness over time and refines pruning decisions to maintain competitive advantage.

Robotic construction and maintenance

The practical aspects of the VinPRO system lifecycle management presents significant challenges that directly impact the economic viability of automated pruning solutions. Robot assembler companies require manufacturing processes that balance cost efficiency with quality standards while ensuring reliable

component sourcing and assembly procedures. Robot maintainers need systems designed for efficient operations and servicing, with accessible components to minimize downtime and complexity. Hardware and services supplier companies must provide reliable components and technical support.

Since robotics agricultural systems operate in demanding environments that accelerate wear and component degradation, the modular design philosophy becomes essential for enabling cost-effective maintenance and component replacement. To compete in agricultural markets, robotic equipment must meet established cost benchmarks and serviceability standards [34].

The first requirement is the business specification that maintenance on the system should cost less than the system itself, ensuring sustainable operational economics over the system lifecycle. This cost constraint prevents excessive maintenance expenses, maintaining the economic benefits of automation while establishing realistic expectations for ongoing operational costs.

Another requirement is that the maintenance of the system should not be harmful for the robot maintainer, representing both a regulatory and individual safety specification. This ensures appropriate safety protocols and equipment design that protect service personnel during maintenance operations.

The technical and business requirement that recovering from system failure should take less than 1 hour minimizes operational disruptions. Also on the technical side, different versions of the system should be available to adapt to all situations, and the robot should be able to perform cuts without damaging the plant to ensure the same quality standards as expert human workers.

Energy consumption

Energy efficiency is a critical concern that impacts both economic aspects and technical stability in agricultural automation systems. Large vineyard owners require energy-efficient operations that minimize operational costs while ensuring suitable power systems for extended seasonal operations, while regulatory institutions must ensure instead that agricultural automation does not stress the electrical grid or make the energy distribution systems unstable in rural and difficult-to-reach agricultural regions. Particularly, when multiple robotic systems operate simultaneously during peak periods, concentrated and remote demands lead to complex needs.

Therefore, the requirement that the system should not create instability in correlated infrastructures, such as power grids, represents both a technical and regulatory specification that ensures the VinPRO system does not compromise broader electrical infrastructure reliability while enabling the large-scale expansion of robotic agricultural systems.

Quality and price ratio

The economic value proposition of automated pruning systems must demonstrate clear benefits across the entire agricultural value chain, while maintaining – or, ideally over time, improving – product quality standards. Consumers expect consistent quality at reasonable prices; retail stores seek reliable product availability and competitive pricing, which automated efficiency can potentially deliver; vineyard owners require systems that enhance operational economics without compromising quality; and vineyard business experts look for comprehensive cost-benefit analyses that demonstrate the long-term economic viability of adopting the technology.

To justify significant capital investments, agricultural automation must prove its worth through measurable improvements in cost efficiency, product quality, or – preferably – both. The derived requirements address these two aspects: first, both a technical and business-oriented requirement that product quality must not decline after system implementation; and second, arguably the most important business requirement, that the system must be economically rewarding for vineyard owners, delivering a clear return on investment within a reasonable time frame.

Environmental needs

Environmental impact is a common concern in the adoption of automation, extending beyond immediate vineyard operations to encompass broader community and regulatory considerations. Neighbouring communities require assurance that agricultural automation will not negatively affect noise levels, local environmental quality, or employment opportunities, while regulatory institutions must ensure that automated systems contribute to – rather than detract from – environmental sustainability goals. The integration of robotic technology can offer benefits through optimize resource utilization, but it also raises concerns about electronic waste and community impact.

The environmental requirements address multiple sustainability dimensions. First, there is a regulatory requirement that local noise levels must not exceed legal limits, thereby protecting neighbouring communities. Second, both a regulatory and social commitment is addressed by the specification that at least 25% of the system’s energy must be sourced from renewables. Finally, a social requirement calls for the creation of new job opportunities, countering widespread fears of mass job loss and ensuring that automation contributes positively to local economic development.

4 State of the Art

4.1 Classic mechanical pruning techniques

According to [4], hand pruning still accounts for 75% of the yearly labour demand but this number can be lowered down through mechanization by 50% to 90% depending on the applied technique and the extent of hand cleaning job left to the worker.

Since the 1970s, several mechanical pruning techniques have been developed and consequently many studies about the effects on grape composition and health status of the plants were carried out.



Figure 1 Tractor with a mechanical pruner. CC Landini <https://www.landini.it/as/the-mechanical-pruning-of-vineyards/>

Mechanical pruning involves the usage of tractors carrying a machine called pruner, which performs straight cuts on the vineyard, followed by human intervention to refine the errors and collect cut-out pieces. This approach can be considered as a facilitated manual pruning as the results do not deviate much from those of the more traditional technique in relation to the number of fruit/spur heads left, load of buds and number of cuts made. There are obviously some differences, especially in the quantity of remaining buds resulting in a greater number of bunches, which are more sparse and less susceptible to attacks by grey mould. [5]

When no manual intervention is performed after the mechanical cut we talk about **full mechanical pruning**, that can be carried out in less than 5 hours per hectare. This technique is obviously faster but less selective and precise, resulting in worse-quality final product.

Another widely adopted method is **mechanical pre-pruning**, where a machine removes the bulk of one-year-old wood prior to a final manual pass. This hybrid approach significantly reduces the time and skill required for manual pruning while allowing for more precise selection of fruiting spurs.

[4] applied these methods and compared them in a five-year study on Trebbiano Romagnolo grapevine variety, a cultivar known for its low basal bud fertility. Their results confirmed that even in such challenging varieties, mechanical pruning (both with and without manual follow-up) can be a viable alternative. Productivity increases were modest (less than 15%), due to compensation factors like bud and shoot regulation, but grape composition remained comparable to that of manually pruned vines. The study also highlighted that full mechanical pruning without manual correction was feasible and delivered results similar to light manual follow-up. Furthermore, economic sustainability was achievable even in small vineyards (around 3 hectares), supporting the broader adoption of mechanized solutions.

Importantly, mechanical pruning has been linked to improved vine health, particularly in reducing wood disease incidence. A study by the French Wine and Vine Institute (IFV) observed up to three times fewer symptoms of Esca and related trunk diseases in machine-pruned vines, possibly due to fewer large pruning wounds and a more consistent wound pattern.

4.2 Autonomous robotic pruning

As of today, autonomous robotic pruning is a field that is not rich in research and development, with a limited number of papers published in the last decade.

[6] analyses the papers from 2014 to 2024, with the goal of creating an up-to-date review of the literature. This analysis highlights that most of the papers target apple trees (31.6%) as the main tree species to prune, just followed by grapevines (24.6%).

Autonomous pruning is not a one-shot task but a pipeline of different tasks, each with its own State of the Art and Technology Readiness Level.



Figure 2 Steps in the autonomous pruning process

- Tree perception: involves all the algorithms that are able to identify and spatially locate tree branches, buds, fruits, leaves and their typology.
- Tree skeletonization: consists in leveraging the previously obtained information to estimate the tree structure in a graph-like representation.
- Cutting point estimation: identifying the points on the tree skeleton that need to be cut, using heuristics or learning-based approaches.
- Manipulator design: the manipulator should be able to move and operate in a potentially intricate environment, therefore needing several degrees of freedom, and should also be fast moving and precise.
- End effector design: the end effector must be carefully chosen in order to be able to perform the cut effortlessly and precisely, depending on branches size and robustness.
- Planning and control: consist in calculating a trajectory for reaching the pruning point avoiding collision with existing occlusions.

Therefore, it is difficult to find end-to-end solutions but rather separate solutions for each task, with the first tree dominating in the number of available papers probably due to the already existing and perfectly working solutions for the latter tasks that need no big improvements.

4.2.1 Tree perception

The first step in autonomous robot pruning is visually detect and segment tree structure. Several algorithms were developed over the years, with the first ones relying on handcrafted features and classic image processing pipelines, while the most recent leverage the advancements in machine and deep learning.

First solutions used 2D RGB images and standard computer vision algorithms.[7] used morphological features and multi-class Support Vector Machine (SVM) to detect citrus branches, [8] segmented apple tree branches using Contrast-Limited Adaptive Histogram Equalization (CLAHE), while [9] combined RGB images with laser scanning to detect apple trunks.

Limitations of 2D solutions were evident, causing imprecisions in all that tasks that require precise special localization like pruning. For this reason, new 3D technology like stereo vision and Time-of-Flight (ToF) cameras, started to be adopted, bringing huge improvements in special accuracy. For example,[10] used a stereo camera to collect RGB and depth info,[11] reconstructed vine structures in 3D.

The advent of deep learning methods marked a big improvement in the accuracy of detection and segmentation, thanks to better generalization capabilities. Most of these methods are based on Convolutional Neural Networks (CNNs) that, thanks to their translation invariance and scalability, are well-suited to handling variations in branch positions and lighting conditions.

Region-based CNN (R-CNN) and its enhancements Fast R-CNN, Faster R-CNN, Mask R-CNN have been adopted in several solutions. These architectures use Region Proposal Network (RPN) to identify potential objects, then each region is processed by the CNN. For instance: [23] used Mask R-CNN and Cascade Mask R-CNN to detect and segment apple trees, [12] compared two models of a Mask R-CNN to detect vigorous leaders in sweet cherry trees, [30] compared different backbones in the context of segmenting multiple grapevine branch classes, [32] performed grapevine segmentation, [18] used a tiny Mask R-CNN in instance segmentation of branches and fruits in guava crops. Faster R-CNN has been used by [14] to detect the visible part of the cordon for green shoot thinning in grapevine and by [42] to identify apple trees' bounding boxes from UAV images.

Among the other used networks, we can find: SegNet, a fully convolutional network for semantic pixel-wise segmentation consisting of an encoder-decoder structure, has been used by many; SOLOv2, a simple, direct, and fast framework for instance segmentation; PSP-net, which introduces pyramid pooling modules; YOLO, an object detection model that predicts bounding boxes through a regression-based approach.

All the described approaches use 2D RGB data. Among the few ones that exploit 3D information, [13] used a 3D point cloud obtained through two cameras with a ToF sensor.

4.2.2 Tree skeletonization

The second step, after detecting and segmenting the tree, is the reconstruction of its structure, either in 2D or 3D. A wide range of techniques can be applied, from simple fitting-based methods to complex multi-stage algorithms. The simplest approaches rely on polynomial fitting applied to the segmentation mask, as done in [10], [14], [12], [11] where the centrelines of branches are estimated directly from the detected silhouettes.

More advanced solutions include the fast parallel thinning algorithm of [15], which iteratively removes boundary and corner points in two sub-steps: first eliminating southeast boundary and northwest corner points and then removing northwest boundary and southeast corner points, preserving pixel connectivity and producing a one-pixel-wide skeleton.

Building on this, more sophisticated methods reconstruct 3D point clouds from multiple viewpoints or depth sources, followed by registration with algorithms such as RANSAC, Iterative Closest Point (ICP), or Normal Distributions Transform (NDT). Once the merged point cloud is obtained, clustering methods like DBSCAN or Kd-tree structures are used to segment trunk and branch regions before extracting the skeletal graph. Graph-based approaches can derive connectivity heuristically, as in [16], where a minimum spanning tree is computed using Delaunay triangulation and Dijkstra's shortest path, or by incorporating prior biological knowledge into the reconstruction, using branching thickness, curvature, and growth direction as constraints.

Geometry-fitting approaches model branches and trunks as primitive shapes, most often cylinders or spheres, and fit them using RANSAC or PCA [17], [18], [19]. Other works adopt medial axis transforms [20] or Laplacian-based techniques [13], which refine the skeleton through eigenvector analysis and path consistency checks. Space colonization algorithms [21] are also common, where skeleton points grow toward regions of high point density, producing biologically plausible structures.

Learning-based approaches have also been introduced. For example, [20] employ PointNet++ to learn skeletal points directly from the point cloud, coupled with a greedy connection algorithm, while [22] use vector field prediction to infer node dependencies and connect them using shortest-path optimization on a resistivity graph. Hybrid methods combine classical processing with machine learning, such as [23],

which integrates YOLO-based bud detection with thinning-derived skeletons to reconstruct 3D grapevine plants.

4.2.3 Cutting point estimation

Once the tree skeleton is detected and reconstructed, it is necessary to estimate the pruning points where to perform the cut. Generally, the localization of pruning points can be seen as a combinatorial optimization problem. [24] proposed two discrete differential evolution (DDE) methods, optimized through an empirical model of light interception, to simulate pruning and identify optimal cuts. These methods rely on either a path-based or a set-based approach, operating within a discrete search domain under specific mutation operators. A critical step in this process is defining quantifiable evaluation metrics; for this purpose, [25] introduced the DOLPHIN set, which assesses cane quality through parameters such as diameter, orientation, length, health, internode length, and node count, enabling consistent pruning decisions across seasons and even in other crops. In some cases, the decision policy is simplified, as in [26], where primary jujube branches are shortened by one-third annually to limit nutrient consumption, basing pruning solely on branch length. Other works adopt a multi-feature approach, incorporating parameters like length, position, angle from the head, or distance below support wires, with cost functions tuned on expert-labeled vines to quantify pruning quality [27].

For grapevine, a more complex pipeline is often required, starting with semantic segmentation and branch type identification, followed by bud detection [28][29], since the cutting position is strongly influenced by bud placement. Examples include [30], who locate potential cuts between nodes or cane bases, and [23], who combine bud localization with depth data to select the key pruning point on the skeleton and extract its 3D coordinates.

Alternatively, pruning points can be estimated without explicit structural reconstruction by directly training on expert annotations. [31] applied a Faster R-CNN to detect pruning regions in vineyard images, later refining their model [32] with additional vineyard data and categorizing results by wood type, orientation, and visibility, highlighting that occlusions and poor visibility remain major factors affecting detection accuracy.

4.2.4 End effector design

Grippers implemented in the design of pruning actions generally rely on 6-DOF manipulators equipped with six prismatic joints. These joints represent the primary cause of execution delays, as the robotic arm typically requires a wide range of joint rotation to maneuver the cutting tool effectively. According to the literature, robotic arms with more than five degrees of freedom are often designed by studying the human operator's body movements during the pruning process itself.

These robotic systems are typically composed of three main parts:

1. A base support equipped with a rotary joint,
2. A main body containing a prismatic joint,
3. A distal arm segment that includes the elbow joint, a rotary joint associated with the forearm, and the forearm itself.

The end-effector is mounted on the terminal part of this structure.

Selecting an appropriate cutting tool requires a thorough analysis of environmental conditions. Reaching the target cutting area often involves navigating through densely vegetated regions with branches and foliage. Therefore, the design must address both mechanical and spatial constraints, such as size, shape, weight, and maneuverability of the system.

The primary cutting techniques employed in end-effector design are shear cutting and saw cutting. The former allows for a stable operation, producing clean and precise cuts; the latter is preferable for thicker

branches, although it generally results in a rougher cut surface. In viticulture (grapevine pruning), shear cutting combined with electromechanical actuation is typically preferred. [6]

Among the studies focused on the development of robotic tools for automated pruning is the work by Zhang et al. (as stated in [6]), which proposed a blade mechanism consisting of a motor drive, planetary gearbox, transmission system, a mobile cutting blade, and a stationary cutter, developed specifically for jujube plantations. The robotic arm is responsible for positioning the end-effector near the target branch. Once the photoelectric sensor detects the branch between the blades, the motor is triggered to close the blades. Considering that the average diameter of a jujube branch ranges from 5 to 20 mm, the cutter mouth must be designed for maximum entry efficiency, with an optimal opening angle between the fixed and moving blades of approximately 40° .

For cherry tree pruning, You et al. (as stated in [6]) developed a blade system mounted on a pneumatically actuated 4-bar Cartesian structure. This system was later replaced with a battery-powered bypass pruner, controllable via serial communication. An Intel RealSense D435 RGB-D camera was mounted above the blades at a 10-degree tilt to allow visualization of the upper blade. The system was specifically designed for branches with diameters greater than 32 mm.

Silwal et al. (as stated in [6]) investigated how the mechanical properties of grapevines vary depending on nutrient and water intake. Their findings included evidence that dead vine branches require a significantly higher cutting force.

The issue of required degrees of freedom was addressed by Zahid et al. (as stated in [6]), who developed a 3-DOF end-effector as an extension of a Cartesian manipulator with three prismatic joints. The structure consists of three motors with orthogonal rotational axes, terminating in tool designed for cutting branches with diameters less than 30 mm.

4.2.5 Planning and control

Once the point in the three-dimensional space where the cut must be performed is located, it is necessary to plan the trajectory of the end-effector to reach the point and execute the cut. Once the path estimation is accurate, a collision-free trajectory must be computed, taking into account natural constraints. Teng et al. (as stated in [6]) considered a hierarchical control strategy for highly redundant manipulators to perform complex actions in Cartesian space. The primary tasks are prioritized, while the execution of secondary constraints is designed concurrently. A task-space segmentation allows simultaneous actions to be performed, with joint velocities computed through an incremental Jacobian matrix. The vision system is responsible for detecting pruning points, followed by quintic polynomial interpolation to allow for a smooth trajectory to follow. The Rapidly-exploring Random Tree (RRT) strategy enables trajectory planning that minimizes collisions and increases the quality of the path followed by the manipulator: the cutting tool is brought close to the target by moving the base, and only from that point on is Cartesian planning used for the final alignment to the target. According to You et al. (as stated in [6]), this practice minimizes large joint movements. An advanced version of the RRT algorithm was developed by Chen et al. (as stated in [6]): it features goal-biased sampling and adjustable trajectory step-size, coupled with B-spline trajectory interpolation. This results in a shorter trajectory with a 55% reduction in planning time. Another emerging frontier involves the use of Reinforcement Learning (RL). Instead of using model-based approaches, control policies are created with the help of machine learning. Trial-and-error simulations allow the system to identify cutting points using data from the camera. This strategy improves performance in dynamic environments by combining machine learning with reactive control.

4.2.6 End-to-end solutions

[33] is one of the few end-to-end vine pruning solutions available in literature. The system, called Bumblebee, is composed of a vision system, a ground robot, a 7-DoF manipulator and ad-hoc pruning algorithms. Among the different techniques used, it is worth to note the usage of an active light stereo camera system that allows operating the robot with different lighting conditions. Cut-point detection follows graph-based algorithms. Producers claim to reach a pruning accuracy of 87% but with a speed of 213 sec/vine, much slower than human experts.

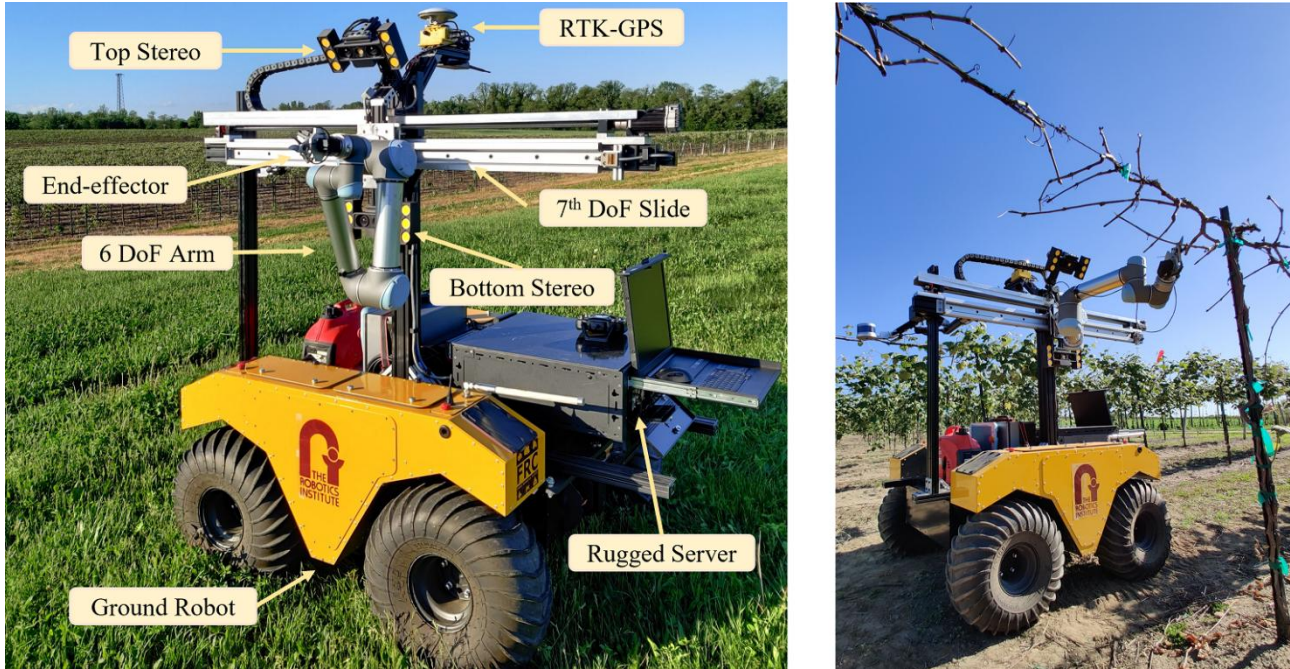


Figure 3 The solution proposed by Silval et al.3

[11] created a mobile robotic platform that straddles vines and creates a 3D model of each vine, thanks to the use of trinocular stereo cameras. These cameras, together with 14 high-power LEDs, are mounted on one side of the platform, while the other one is painted blue to aid the segmentation of the vine from the background. A 6-DoF robotic arm is used to perform pruning, following collision-free paths after identifying the pruning cuts thanks to the reconstructed structure. Pruning each vine takes around 2 minutes, comparable to human experts.

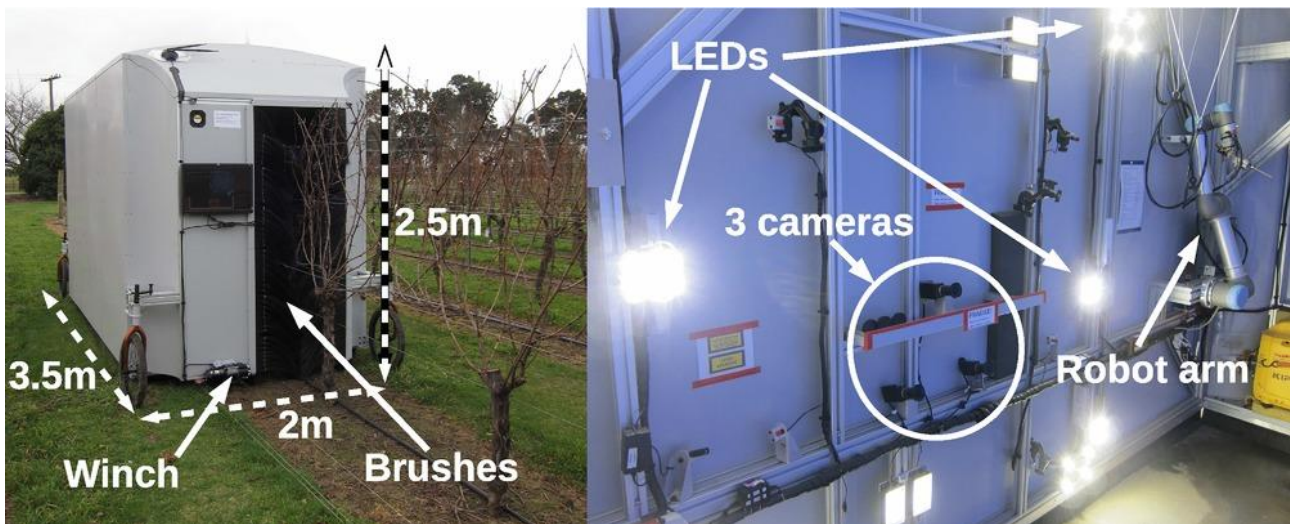


Figure 4 The solution proposed by Botteril et al.4

5 The Solution

5.1 Vine Nodes detection

In order for the robot to see the nodes and branches of the vine, we implemented a Deep Learning pipeline consisting of 2 stacked Hourglass Networks, typically used in human-pose estimation to detect nodes, branch types and branch directions. We inspired our methods and code on the work proposed by ViNet. The paper addresses complete grapevine structure estimation from a single RGB image: detect key nodes on the plant, infer branch types and directions, and reconstruct a tree (in the graph sense) that matches viticulture practice and supports pruning decisions. The approach draws on bottom-up human pose estimation: node “landmarks” + limb-like vector fields, adapted to unknown counts and patterns of branches/nodes in real vines. The paper’s core contributions we replicated are: a stacked-hourglass CNN that predicts node heatmaps and branch vector fields; a shortest-path over a resistivity graph that guarantees a valid tree. We took the same fully annotated public dataset for benchmarking and as a starting point to our experiments.

5.1.1 Dataset description

The dataset contains 1,513 single-plant images (one trunk per image), captured in three fields across French regions with blue or white artificial backgrounds and varying pruning styles. Resolution is typically around 4032×3024 . Despite single-plant framing, overlapping neighbor branches may appear.

Each image is exhaustively labeled with (a) five branch types: Trunk, Courson, Cane, Shoots, Lateral Shoots; and (b) four node types: Root crown, Branch nodes, Pruning cuts, Growing tips; plus explicit parent–child dependencies along branches. The trunk is the oldest wood; Courson and Cane emerge from it; Shoots grow from those; pruning decisions rely on accurate localization and ordering of early Shoot nodes (buds), which is why the metric later focuses on Courson and first Shoot nodes. The train-test split was: 1,255 train / 258 test. The dataset is publicly available for research under CC BY-NC-SA.

For each node category $\mathbf{c}$, ViNet encodes supervision as a 2D heatmap $\mathbf{M}_c$ at quarter input resolution. Each GT node contributes a Gaussian blob centered at its location; the map value is the maximum across nodes of that class, with σ chosen empirically to set spatial spread. For each branch category $\mathbf{b}$ a 2-channel vector field (x/y) encodes local branch direction along every limb segment (between consecutive nodes of that branch). Pixels within a band of width W_{limb} around each segment carry the unit vector from parent to child; elsewhere the field is zero. W_{limb} is set to cover typical branch thickness and stabilize association.

5.1.2 Network (Detection Step)

ViNet uses a Stacked Hourglass (SHG) architecture: a shallow feature extractor (7×7 conv stride 2 + max-pool + residual blocks) followed by two hourglass sub-networks that downsample/upsample features across five scales with skip connections. Each residual block uses a 1×1 – 3×3 – 1×1 pattern with instance normalization and ReLU (instance norm replaces the original Hourglass BatchNorm). All convs in the HGs use a constant K channels. The output tensor is a stack of heatmaps (nodes and branch fields) at $1/4$ input resolution. The second HG refine node/branch maps using context from the first, improving disambiguation when nodes are close to one another or near textured bark. The authors found 2 HGs a good trade-off; adding more brought little gain.

5.1.3 Training losses and implementation

Losses are applied after both hourglasses to mitigate vanishing gradients and encourage progressively refined maps. The objective is the ℓ_2 (Euclidean) distance over all predicted vs. GT heatmaps/fields ($L = L_1 + L_2$) with GT blurs ($\sigma=40$) to match the expected sharpness at each stage.

Images are randomly rescaled so the shorter side is in $[1100, 1300]$, then random 1024×1024 crops are taken. Augmentations include random rotation ($\pm 15^\circ$), horizontal flip ($p=0.5$), and rescale/crop jitter. Batch size is 1, favoring instance normalization for stable training. Implementation is in PyTorch.

Adam with initial $lr=1e-3$, 0.9 decay every 5,000 iterations; training from scratch for 386 epochs.

5.1.4 Post-processing: node coordinate extraction

From each node heatmap M_c , we normalize by the local maximum map within a neighborhood of radius $d = \alpha \cdot Wh$ (image-size-relative); values below $\tau_n = 0.5$ are clipped. We then threshold at $\tau_m = 0.97$, to label the blobs and find the 2D node coordinates corresponding to the blobs' centers.

We build a resistivity graph $GR = \langle N, E \rangle$ where vertices are the detected nodes; for each node n , edges connect to compatible parent types within a radius $r = 0.2 \cdot Wh$. If none are found, we relax and link to the nearest compatible parent. This keeps the graph sparse and focused.

For candidate edge $e = (nc, np)$ with vector $vcp = pc - pp$, we sample along the straight line between nodes and integrate the dot product between the predicted branch vector field (at the parent's branch type) and the unit direction $vcp / \|vcp\|$. The resulting alignment score $A \in [0, 1]$ is 1 when perfectly aligned. The resistivity is $Re = (1 - A) \|vcp\|$, so edges that are short and well aligned with the field cost less.

Finally, we compute Dijkstra shortest paths from every node to the Root crown; the union of these parent links yields a valid tree (one parent per node) even when some fields are noisy. Poor fields can cause "shortcuts" near the root, but the output remains a tree by construction.

5.2 Pruning points selection

After the nodes have been successfully detected, the goal is to establish a pruning policy by selecting which parts of the plant should be cut. The plant structure is represented as a tree, where each node encodes its location in pixels, its predecessors, successors, and its type.

In this context, **predecessors** of a node are those positioned further along the same branch, closer to its extremity. Conversely, **successors** represent nodes closer to the trunk. Formally, if node A is a predecessor of node B , then B is a successor of A . Each node can have:

- **0 predecessors** (end of a branch),
- **1 predecessor** (intermediate point along a single branch), or
- **multiple predecessors** (branching point where multiple sub-branches originate).

Using this structure, we can identify **partition nodes**, defined as nodes with more than one predecessor. Partition nodes mark the start of a branch division. The pruning strategy is then based on selecting partition nodes of interest. However, partition nodes that have other partition nodes as direct or non-direct successors are excluded. The reason is that if a branch is pruned closer to the trunk, cutting it again further away is redundant.

Since we do not aim to cut precisely at partition nodes, we instead target the **midpoints** between a node and its predecessors. These midpoints are practical cutting points: they lie on the branch between two

structural nodes, ensuring a cut is placed along the wood rather than directly at a branching node. This approach relies on the assumption that nodes are not too far from one another, making the midpoint a reasonable approximation of a real cutting location.

Finally, after the pruning points have been selected, **one branch must be preserved for next year's yield**. The chosen branch type is the **cane**, as canes are one-year-old shoots that will bear fruit in the following growing season. Keeping a cane ensures that the plant retains productive wood while the older or less promising branches are removed. Other types, such as lateral shoots or short shoots, do not have the same fruit-bearing potential and are typically pruned away to optimize yield.

5.3 Robot control

In this subchapter is illustrated the whole control part of the robotic arm, camera and microcontroller, which is integrated into the whole system maintaining the compatibility with the other work parts developed mostly with the pruning policy and the mechanical ones.

The main objective of this part is to interface with hardware components like: robotic arm, end effector, Arduino and camera sensors. Each and every component is connected in different ways to the portable computer which is controlling and synchronizing the actions of the whole system.

The main output of this part is contained in a GitHub repository, which contains all the packages and launch files. Basically, they consist in C++ and python code to be run inside the Robot Operating System (ROS) environment (further details in the specific section).

In the whole picture the objective was to write a software that would keep as main input the cutting points identified after the plant's structure analysis and perform the cut of the proper branches in the actual plant.

The idea behind the overall implementation is to divide the whole bigger task into smaller subtasks, use different packages for each one and allow for their coordination and synchronization using a standard communication interface provided by the used software. It is important that those packages have a proper dimension, maintaining them too large will be a mess when a new feature has to be implemented making it poorly scalable, while very small packages will bring disorder and cause a communications overhead.

All the code developed in this section can be accessed through a specific GitHub repository[34].

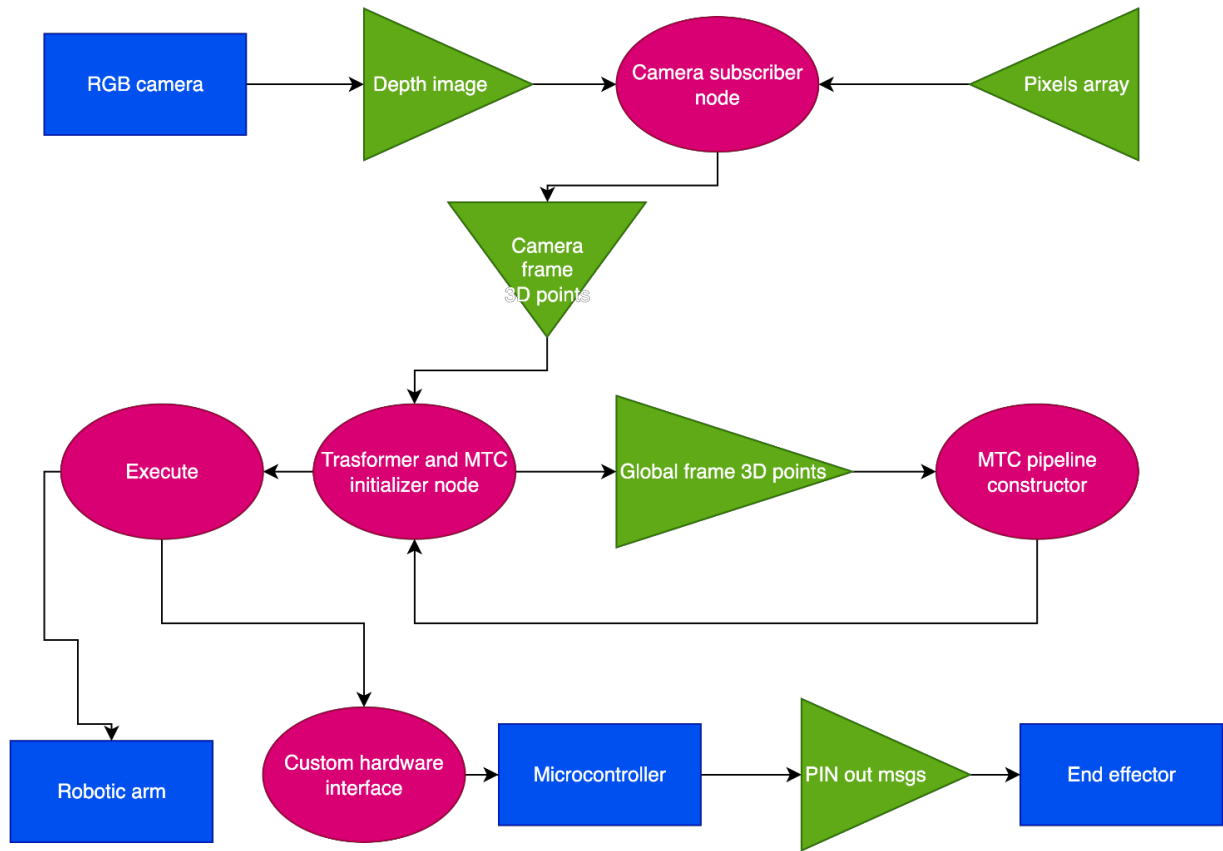


Figure 5 ROS 2 processes and topics architecture for the control part solution

As you can see from the image above the packages are various and each has their own communication topic with the others. Each of them is deeply explained later in this document, here are their basic functionalities:

- *Camera subscriber node* acquires the depth images from the camera and pixel coordinates of the decided cutting points and transforms them into space coordinates to be passed to the proper movement pipeline. The output points are in the camera frame.
- *Transformer and MTC initializer* receives the points and transforms them into the global frame, and then interacts with the MTC pipeline builder and launches the execution of the planned sequence
- *MTC pipeline constructor* sets up the stages of the overall task and computes the inverse kinematics needed to reach the cutting points
- *Scissors hardware interface* is managing the communication and synchronization of the pipeline with the microcontroller.

5.3.1 Software: ROS 2 humble [35] & Moveit 2[36]

In this project, the management and coordination of all subsystems were handled through ROS2. This choice was deliberate: the system required an infrastructure capable of ensuring stable and responsive communication between the Kinova Gen3 Lite robotic arm, the vision system based on an Intel RGB camera, and the motion planning software. With its distributed, node-based architecture, ROS2 allowed the different parts of the project to be developed and integrated in a modular way, avoiding the need to rewrite entire components whenever one part was modified. Its native compatibility with the selected hardware, combined with the ability to handle high-frequency data exchange, was essential to ensure that the information from the camera reached the arm's controller without delays or loss of precision.

Within this ecosystem, MoveIt played the role of motion planning and execution tool. The robot's task—precisely reaching a cutting point among the vine's shoots without touching the parts of the plant that must be preserved—requires extremely accurate trajectory control. MoveIt made it possible to transform

the three-dimensional coordinates provided by the vision system into movements that the Kinova arm could actually perform, computing safe, collision-free trajectories even in the presence of complex and variable geometries. Its simulation engine also allowed movements to be tested in a virtual environment, like *rviz*, before running them in the field, reducing risks and optimizing trial time.

The integration between ROS2 and MoveIt formed the core of the control system. ROS2 managed communication and data synchronization, while MoveIt provided the intelligence needed to turn visual information into concrete, precise, and repeatable actions. Together, they enabled a workflow in which the image recognition routine detects the position of the pruning point, processes the data and sends it to the motion planner, and the robotic arm executes the operation with the precision required in an agricultural application.

In the context of the vineyard pruning robot, MoveIt 2 served as the key software component for motion planning and execution. While ROS2 provided the communication backbone connecting all subsystems, MoveIt 2 was responsible for determining *how* the Kinova Gen3 Lite robotic arm should move in order to carry out the pruning task safely and precisely. This role was far from trivial: the environment in which the robot operates is unstructured, with vine shoots arranged in irregular and variable ways from plant to plant. The cutting tool must be guided with millimetric accuracy, avoiding accidental contact with parts of the plant that should remain intact.

MoveIt 2 processed the target positions identified by the Intel RGB camera and the image recognition system, converting them into feasible arm poses and smooth trajectories that respected the robot's kinematic constraints. Thanks to its integrated collision-checking capabilities, the planner could ensure that each planned movement avoided both the plant itself and any mechanical self-collisions of the arm. This was crucial in preventing damage to the vines and ensuring the repeatability of the pruning process.

Another key advantage of MoveIt 2 was its ability to adapt trajectories in real time. When the visual system detected a target slightly displaced from its initial position—due to environmental conditions or plant movement—MoveIt 2 could recalculate the trajectory dynamically, allowing the arm to reach the updated position without restarting the entire operation. This responsiveness was essential in an outdoor agricultural scenario, where environmental variability is the norm rather than the exception.

A particularly relevant feature for our application was the **MoveIt Task Constructor (MTC)**. This framework within MoveIt 2 allowed us to break down the pruning operation into a sequence of modular, goal-oriented subtasks, such as approaching the target, orienting the tool, executing the cut, and retracting the arm. By structuring the process in this way, we could design and test each phase independently, while ensuring that the entire sequence was consistent, feasible, and optimized for both safety and efficiency. Although the technical details of MTC will be discussed in the next section, it is important to note here that this approach was fundamental in translating high-level pruning strategies into a series of reliable and repeatable robot actions.

In short, MoveIt 2 acted as the decision-making layer for the robot's movements, bridging the gap between high-level pruning objectives and the precise joint motions needed to achieve them. Its integration with ROS2 created a control loop in which visual data was transformed into safe, optimized, and repeatable motion sequences, perfectly suited to the precision and adaptability demands of vineyard pruning.

5.3.2 Software: Task constructor [37]

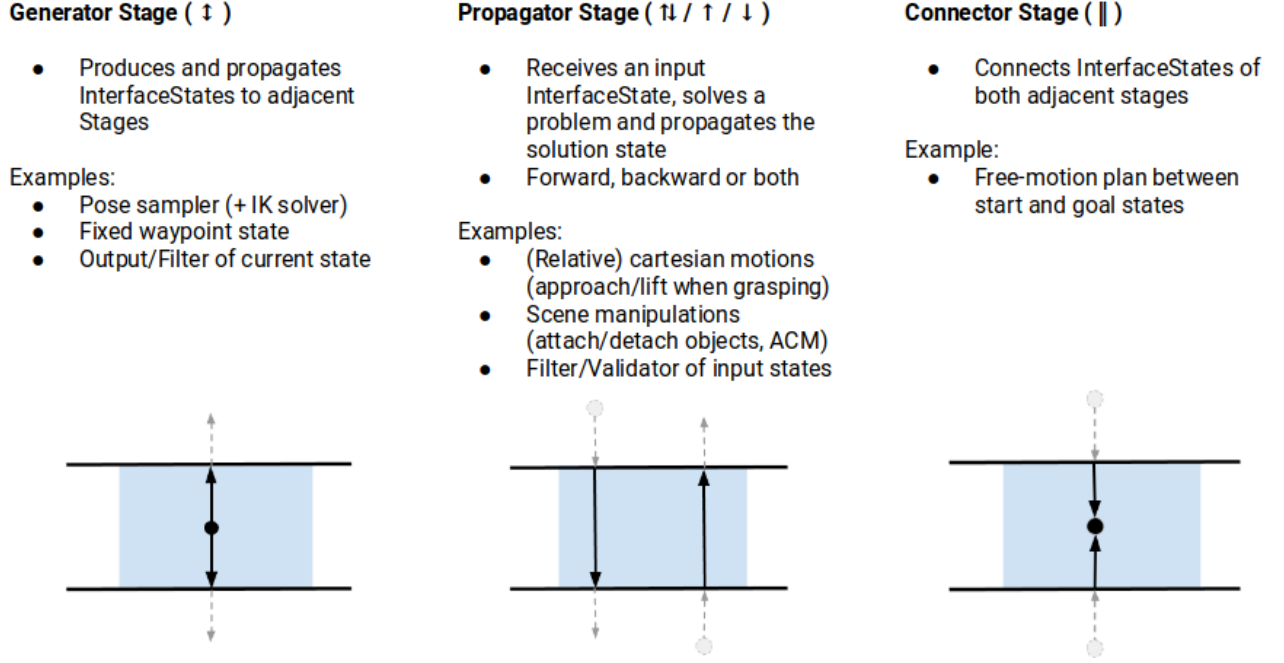


Figure 6 Overview of different types of stages available in the MoveIt Task Constructor utility

In our vineyard pruning project, the **MoveIt Task Constructor (MTC)** was adopted as the framework to translate the high-level goal (identify a pruning point and remove the shoot) into a sequence of planned and verifiable actions. In simple terms, MTC is not a single monolithic planner but a library for breaking down complex tasks into multiple smaller subtasks (called *stages*), letting MoveIt solve each of these sub-problems, and passing solution hypotheses between stages in a structured way. This approach allowed us to reason about the entire pruning operation as a modular pipeline rather than as a single, opaque planning call.

The conceptual architecture of MTC is built around three main elements: the *Task*, the *Stages*, and the *InterfaceState* which links partial solutions together. A Task is essentially a hierarchical container that can include both elementary stages and other containers; containers can be executed in sequence or in parallel, allowing the design of linear pipelines or more complex compositions (such as concurrent attempts with fallback strategies). Stages themselves are typically divided into generators (which create possible states), propagators (which expand hypotheses forward or backward), and connectors (which attempt to find a trajectory linking two states). Each stage exposes configurable properties (tolerances, planner choice, reference frames) that allow fine-grained control of behavior and constraints.

From a planning perspective, MTC *decouples planning from stage implementation*: stages describe *what* needs to be achieved (for example, a pre-cut pose, a Cartesian approach, a specific tool orientation), while the planners (OMPL, Cartesian path generators, etc.) handle the actual computation of a feasible trajectory that satisfies those constraints. This separation makes it possible to reuse the same stage with different planners (for example, OMPL for free-space positioning and a Cartesian generator for straight-line approach motions) and to combine global and local planners within the same pipeline. In our project, this translated into defining “approach” stages with a Cartesian planner for delicate linear movements toward the vine shoot, while relying on OMPL for larger repositioning movements.

Applied to vineyard pruning, MTC allowed us to model the single cutting operation as a logical sequence of clearly testable phases: acquiring the current state of the robot and the updated planning scene (with obstacles from the vision pipeline), generating candidate poses around the pruning point, planning a controlled approach (often a Cartesian path) to place the tool with the desired orientation, executing the cutting motion, and then retracting the arm. Each phase was defined as a stage with specific properties (e.g., velocity limits, end-effector orientation constraints, selective collision checking). Additionally, MTC supports monitoring and fallback components—for instance, inserting verification stages that discard

solutions with recent collisions or providing alternative routes if the primary planner fails. This structure simplified handling common real-world scenarios in the field, such as plant movement due to wind or minor discrepancies between visual estimates and the actual geometry.

On the implementation side, MTC is primarily used through its C++ API and integrates fully with MoveIt 2 and ROS 2. This made it possible to develop and validate the pruning pipeline before executing it on the Kinova Gen3 Lite: tests in simulation (RViz + Motion Planning Tasks display) allowed us to observe multiple solutions, inspect intermediate hypotheses, and fine-tune stage properties and tolerances before running the actual hardware. For this reason, we followed a workflow of “task development → validation in RViz → deployment on the robot via `ros2_control` / MoveIt controllers.”

From practical experience, MTC offers strong modularity and clarity when designing manipulation sequences, reduces the amount of custom code needed for complex cases, and facilitates debugging (you can see exactly which stage fails and why). The trade-off is that effective use requires careful management of stage properties, robust integration of the planning scene generated from the vision system (positioning collision objects, attaching/detaching when necessary), and—when going beyond the built-in capabilities—the creation of custom stages. In our project, we balanced these aspects by building reusable stages for common pruning phases and reserving custom stages only for exceptional cases.

5.3.3 Software: ROS 2 control [38]

In our system, `ros2_control` was specifically implemented to manage the actuation of the pruning shears through an Arduino Nano interface. Acting as the bridge between ROS 2 and the low-level hardware, `ros2_control` allowed us to define a custom hardware interface that mapped high-level open/close commands into the precise electrical signals required by the cutting mechanism. This setup ensured consistent and responsive actuation, with the added benefit of integrating the shears’ control seamlessly into the overall ROS 2 architecture. By using `ros2_control`, we were able to maintain the same command and feedback structure as other robot components, simplifying system integration and making the control of the pruning tool fully accessible within the same planning and execution pipeline.

5.3.4 Hardware

Due to their high cost, mostly for the arm and camera sensor, these commercial hardware components were not object of any design choice. They have been furnished by pic4ser and YANMAR as they sponsor the project.

In any case they were more than enough to complete the project, here we show their characteristic and why they would be a good choice to complete the project.

Kinova Gen3 Lite[39]



Figure 7 Photo of the Kinova Gen3 Lite arm equipped with a commercial 2 finger gripper

It is the primary manipulator for the pruning robot due to its combination of precision, compact design, and ROS 2 compatibility. Its lightweight construction made it suitable for integration on a mobile agricultural platform without adding excessive structural load, while its six degrees of freedom provided the flexibility needed to maneuver around irregular vine geometries. The built-in joint torque sensors and the availability of official ROS 2 drivers allowed for smooth integration into our control framework, enabling both high-level motion planning through MoveIt 2 and low-level joint control for specific maneuvers. Its repeatability, within millimeter-scale tolerances, was crucial to ensure that the cutting tool consistently reached the exact point identified by the vision system, even when operating under outdoor conditions where minor platform vibrations or plant movements are inevitable.

Arduino Nano[40]



Figure 8 Photo of the Arduino Nano

An Arduino Nano was employed as an auxiliary controller to manage specific low-level tasks that did not require the full computational overhead of ROS 2. In our configuration, it was used to interface with peripheral components such as the cutting tool actuator and certain safety sensors. The Arduino Nano's small footprint, low power consumption, and straightforward serial communication capabilities made it an ideal choice for offloading these control loops from the main ROS 2 system, reducing processing load

and improving responsiveness. By handling time-critical I/O operations locally—such as triggering the cutting mechanism when commanded—it ensured that these actions occurred with minimal latency, independent of the higher-level motion planning cycle.

Intel RealSense D435i[41]



Figure 9 Photo of the Realsense D435i RGB depth camera

The Intel RealSense D435i depth camera formed the basis of the robot’s vision system. This model combines a high-resolution RGB sensor with a stereo depth module and an integrated inertial measurement unit (IMU), enabling both color imaging and accurate 3D point cloud generation. Mounted on the robot to provide a clear view of the vine structure, the D435i allowed the vision algorithms to detect pruning points and reconstruct the surrounding geometry for collision avoidance in MoveIt 2. The inclusion of the IMU also helped improve spatial awareness, particularly when the platform or the plant was subject to slight movement due to environmental factors such as wind. The RealSense SDK and its ROS 2 integration packages facilitated real-time data streaming directly into the perception nodes, ensuring that updated spatial information was always available for planning and execution.

5.3.5 Image pixels to space points transformation

This node, developed in C++, serves as the link between the robot’s visual perception and the control system that performs the grapevine cutting. Its purpose is to transform purely two-dimensional information—the pixel positions identified by the vision system as cutting points—into three-dimensional spatial coordinates that can be used by the robotic arm’s actuators.

The node subscribes to the Intel RealSense depth image stream, in our case already aligned with the RGB channel. By using `image_transport` with a “camera” subscription, it receives not only the image but also the camera’s intrinsic parameters (K matrix, distortion model, optical parameters). At the same time, another subscriber listens to the `/pixel_coordinates` topic, where the image recognition module publishes the coordinates of the pixels corresponding to points of interest. This separation of the video stream from the pixel data avoids embedding vision logic inside the node, keeping it modular and reusable.

When both data sources are available (depth image + pixel coordinates), the node performs the 3D position calculation. At the core of this process is the `rs2_deproject_pixel_to_point` function, which implements the reverse transformation of the camera projection: starting from the image coordinates (u, v) and the depth value at that pixel, it reconstructs the actual coordinates in the camera’s reference frame. It also supports the “plumb_bob” distortion model, correcting for lens-induced optical

distortion—particularly important in agricultural contexts where targets can appear near the edges of the field of view.

Depth values are accessed via `cv_bridge`, which converts the ROS message into an OpenCV structure, allowing direct numeric access to the depth frame. The pixel indices are stored between callbacks in a small buffer (`pixels_`), initialized with sentinel values (-1, -1) to indicate when no valid coordinates have yet been received.

Once the 3D point is obtained, the node prepares its output using the custom message type `custom_msgs::msg::PointsArray`, which has been created by us and defined into the package `custom_msgs`. This format was chosen to enable sending multiple cutting points detected on the same grapevine on a single publication. The message also includes the original image header, preserving timestamp and frame ID—essential for synchronization with the rest of the processing pipeline.

The output is published on the `/detected_goal_pose` topic, which the robot’s control module uses to plan and command the cutting tool’s motion. With this architecture, the flow from perception to decision is clear and traceable: vision provides the “where” in pixels, the transformation node converts it into “where” in real space, and control will handle the “how to get there.”

From a design perspective, this approach was chosen for flexibility and reliability. Working directly with the camera’s calibration data, instead of relying solely on pre-packaged transformations, ensures full control over the process and makes it easier to adapt to new hardware configurations (different optics, cameras mounted at varying distances from the arm, etc.). Furthermore, having direct per-pixel access to depth enables the future implementation of filtering or correction logic tailored to agricultural environments, where reflections, leaves, and branches can introduce noise into the measurements.

The depth camera also provides a point cloud image, but since it might be less precise we preferred to “manually” convert 2D to 3D points inside the node.

5.3.6 Transform node

This node is responsible for bridging the high-level perception data—specifically the 3D cutting coordinates produced by the vision-processing pipeline—with the actual execution of movements on the manipulator using MoveIt’s planning capabilities and the Task Constructor framework.

Upon initialization, the node subscribes to `/detected_goal_pose`, which carries the `custom_msgs::msg::PointsArray` message generated by the perception subsystem. Each message may contain one or more target points that the manipulator must reach, typically corresponding to cutting locations along the grapevine.

The first step when receiving a message is to log and iterate through the points. Each coordinate in the `PointsArray` is expressed in millimeters relative to the camera’s frame, so the node immediately converts these values to meters and then uses the TF2 transform library to convert them from the camera frame into the robot’s `base_link` frame. This transformation is necessary since it ensures that the motion planning operates in a global reference frame consistent with the robot’s kinematics.

For each transformed point, the node also creates and publishes an RViz visualization marker. These spherical markers provide visual feedback of the detected goal positions in the environment, which is critical during debugging and field testing in the vineyard. This visualization step was integrated early in development to quickly verify that perception and TF transformations align correctly, and that the transformation performed by the camera node is consistent, before attempting actual manipulator movements.

Once all points are transformed into global coordinates, the node instantiates a `PlanningTask` object from the `mtc_multiple_points_demo` namespace—a custom wrapper around MoveIt Task Constructor logic. More details are present in the next subsection.

Before running the task, the robot’s URDF model is loaded via `robot_model_loader::RobotModelLoader`. This ensures the planner has an up-to-date kinematic and collision model of the manipulator.

The task initialization (`task.init`) is passed the list of global target points, enabling it to generate motion plans for all required positions in one cohesive sequence. Planning is then attempted with up to 10 attempts, which balances execution speed with robustness in the presence of obstacles or tight kinematic constraints.

If planning succeeds, the node proceeds to execution, commanding the manipulator through the planned sequence of movements. After task completion it loops back to wait for the next set of points.

From a project standpoint, this node plays a critical role in the “decision-to-action” chain. The perception system provides the “where” in space, and this motion-planning module turns that into an optimized, collision-free trajectory that can be safely executed in the unstructured and cluttered vineyard environment. The use of MoveIt Task Constructor here is intentional: it provides a scalable way to plan multi-stage actions, which will be increasingly important if future versions of the robot must handle branch avoidance, repositioning, or tool changes between cuts.

By integrating TF transformations, visualization feedback, and robust planning sequences, this node ensures that every point received from perception can be confidently and precisely reached by the manipulator, closing the loop between sensing and acting in the grapevine pruning workflow.

5.3.7 MTC pipeline design

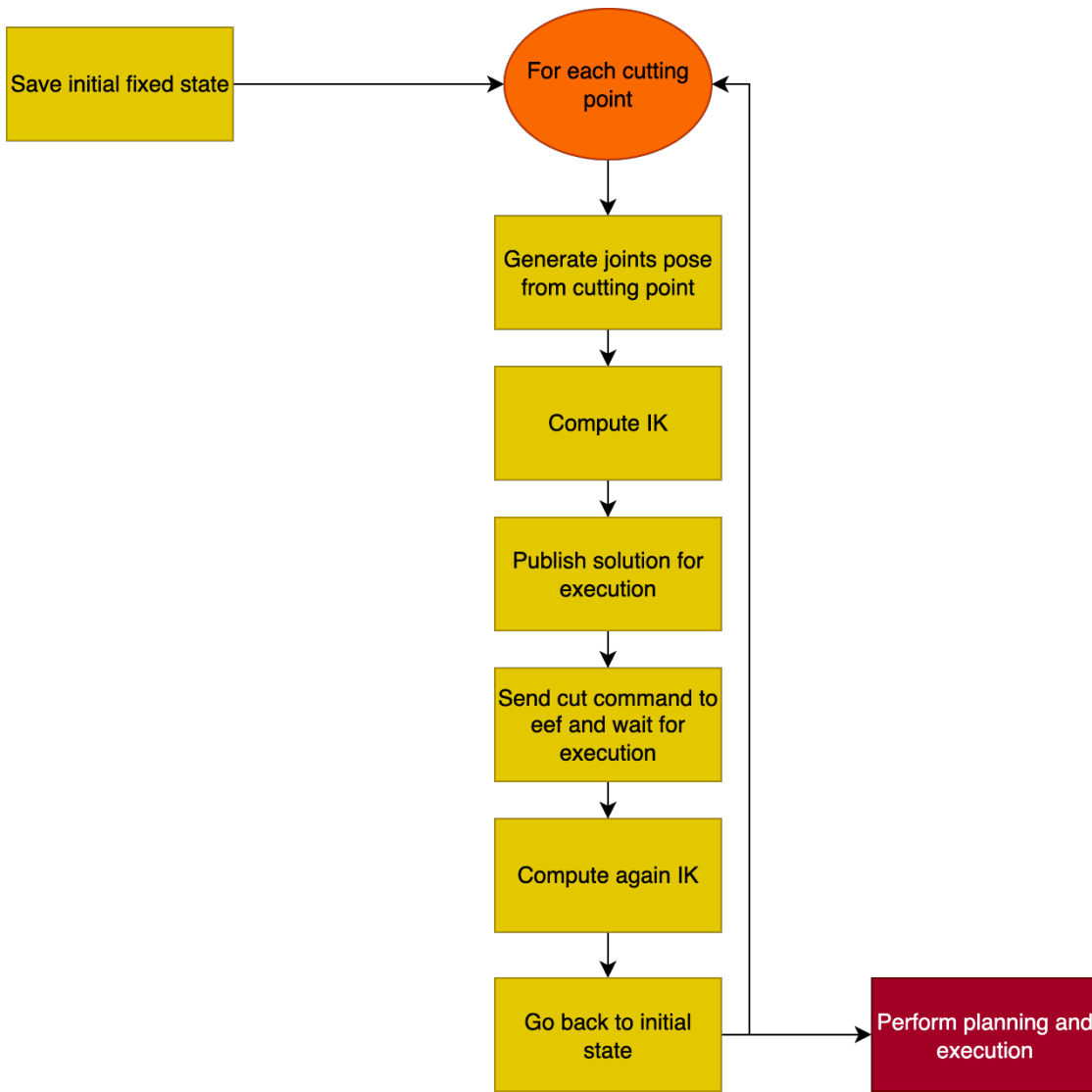


Figure 10 Flowchart of the MTC task implemented in the solution

The motion planning in this node is organized around the MoveIt Task Constructor (MTC) framework, which breaks a complex motion sequence into a series of connected stages. Each stage encapsulates a discrete planning problem, making it easier to handle multiple waypoints, error recovery, and complex motion sequences in a structured way.

The task begins with a **Current State** stage, which captures the robot’s actual configuration at the start of planning. This provides a consistent baseline from which all subsequent motions are computed. From there, a **Connect** stage links the current configuration to the first goal pose using a chosen planning pipeline—in this case, the RRTConnect algorithm. This ensures that even if the manipulator’s starting pose is not already in a collision-free path to the goal, the planner will attempt to find one.

For each target point received from the perception system, the task includes a **MoveTo** stage, which specifies a Cartesian or joint-space motion toward that goal.

When multiple points are involved, these stages are chained together so that the output state of one becomes the input state for the next. This avoids re-planning from scratch for each goal, making the overall process more efficient. Additional stages, such as pre-positioning or tool orientation adjustments, can be inserted into this chain without restructuring the rest of the task.

Finally, the task’s sequence of stages is planned as a whole before any execution begins. This holistic approach means the planner can optimize transitions between points, avoid unnecessary backtracking,

and ensure collision-free paths throughout the entire sequence. If any stage fails, the task reports the failure early, allowing the system to either retry with different parameters or notify a higher-level supervisor for intervention.

In this way, the node leverages MTC not just as a planning tool, but as a modular execution framework that can evolve alongside the complexity of the pruning process, supporting the project's long-term goal of fully autonomous vineyard operation.

5.3.8 Custom hardware interface for Arduino

The package implements a custom hardware interface named `ScissorsController` within the `arduino_controller` namespace, designed specifically to interface a ROS 2 control system with an Arduino that physically operates a pair of robotic scissors used for pruning vines. The controller handles communication over a serial port with precise termios settings to ensure reliable, raw serial communication without parity, flow control, or echo. This low-level serial setup is crucial to achieve real-time, stable communication with the Arduino, which acts as the actuator hardware in this system. During initialization (`on_init`), the controller opens and configures the serial port, handling errors gracefully by falling back into a simulation mode if the port is unavailable, thus maintaining system robustness even without the physical device connected.

The controller exports both state and command interfaces related to the joint it manages, specifically exposing position and velocity state interfaces and a position command interface. This design allows higher-level ROS controllers to command precise position targets for the scissors actuator and read back its current state if available. When activated (`on_activate`), the controller sets up a ROS 2 node and publisher and resets internal state variables like velocity commands and positions to zero, preventing stale data from affecting actuator commands.

The `write` method encapsulates the core communication with the Arduino, responsible for sending commands and ensuring they are successfully executed. It rounds the commanded position to two decimal places and only proceeds if the command is nonzero, reflecting a design choice to avoid unnecessary communication for idle states. The write operation involves sending a specific character command (`'t'`) over the serial port, which triggers the Arduino to execute the cutting action. The method then enters a loop where it waits for an acknowledgment character `'A'` from the Arduino, confirming that the cut was successfully performed. This acknowledgment protocol is vital for synchronization and feedback, guaranteeing that the controller does not advance until the physical action is confirmed. If the Arduino responds with any other character, the controller logs this as a failure and retries, emphasizing robustness and reliability.

The `read` method currently functions mostly as a placeholder, returning success without reading actual feedback from the Arduino.

Deactivation (`on_deactivate`) cleanly closes the serial port to free resources and prevent dangling connections.

Throughout the code, there is careful use of ROS 2 logging (`RCLCPP_INFO`, `RCLCPP_WARN`, `RCLCPP_ERROR`, etc.) to report on the system's state, errors, and important events, which greatly aids debugging and monitoring during runtime, especially when dealing with asynchronous serial communication.

The design of this hardware interface reflects a pragmatic approach: it tightly couples the software commands to the hardware actuator through serial communication, uses simple character protocols for command and acknowledgment, and handles errors gracefully to maintain operational stability.

Overall, this code bridges the high-level ROS 2 control commands with the low-level hardware actuation performed by the Arduino-controlled scissors, providing the necessary interface to command precise movements, confirm execution, and keep the internal software state consistent with the physical device status. It combines standard POSIX system calls for serial port manipulation with ROS 2 idioms, delivering a robust and efficient actuator interface suitable for a real-world robotic pruning system operating in the vineyard.

5.4 Vineyard simulation

To support the development and evaluation of automated pruning strategies without risking damage to physical plants, a fully virtual vineyard environment was created. This simulated testbed remains perpetually available, reproducible, and immune to the irreversible consequences of incorrect pruning actions. It enables rapid iteration of both perception and manipulation algorithms in a controlled setting before deployment on real-world vines. **Plant Construction in Blender**

The generation of realistic grapevine models was divided into two phases: the creation of individual structural components and their subsequent assembly into complete plants. In the initial phase, artist-crafted 3D meshes for trunks, primary and secondary branches, leaf clusters, and grape bunches were produced using Blender’s native modeling tools. While manual design ensures botanical fidelity, assembling these parts by hand into diverse plant geometries proved labor-intensive and error-prone.

To automate assembly, we implemented a recursive, rule-based algorithm within Blender’s Python API. Beginning with a base trunk segment, the routine iteratively attaches compatible components—such as lateral shoots or grapes—according to botanical growth patterns. At each recursion step, geometric and topological checks verify the realism of the plants, ensuring that:

- No two meshes intersect or overlap unnaturally
- Grapes and leaves follow the natural orientation enforced by gravity and the sun
- Branch lengths and bifurcation angles remain within realistic horticultural ranges

By parametrizing component selection and attachment rules, the algorithm produces a varied library of structurally plausible vines. This approach reduces manual effort, enforces consistency across models, and facilitates rapid generation of large, labeled datasets for vision-based training or simulation studies.

5.4.1 Unity environment

An interactive vineyard scene was constructed in Unity to leverage its real-time rendering capabilities and native support for Robot Operating System 2 (ROS2) integration. Within this environment, virtual pruning tools and robotic manipulators can be controlled via ROS2 topics and services, enabling seamless coordination between perception, planning, and actuation modules.

An early concept involved a physics-driven “shears” object capable of arbitrarily slicing plant geometry at runtime. However, dynamically computing mesh cutting and remeshing for complex botanical structures proved prohibitively expensive in real time. As a practical alternative, we predefined a set of cutting planes—both agronomically correct and intentionally suboptimal—directly in the source 3D models.

During simulation initialization, each vine model is segmented along these planes into discrete sub-entities representing the portions above and below each potential cut. At pruning time, the system treats the intact vine as a single object until a specific cut action is invoked. After performing a cut, Unity’s engine:

1. Separates the mesh of the plant into its two sub-meshes.
2. Applies rigid-body physics to each fragment.
3. Allows the upper fragment to be manipulated independently (e.g., dragged or removed).

This hybrid approach delivers interactive pruning feedback with minimal computational overhead, maintains visual realism, and provides precise labels for correct versus incorrect cut locations. It thus

establishes a robust, extensible platform for validating automated pruning algorithms under a wide range of scenarios.

5.5 Mechanical design

Since the robotic arm was provided by Pic4Ser, it was necessary to design a device to perform the function of the end effector. However, the model of the arm used did not allow for the removal of the original gripper. Therefore, a custom-made cover ("glove") was developed to enclose it. On this structure, the following components were integrated: the vision system, the cutting tool, its electric actuator, the cutting control and safety systems.

The main design requirements for the system were:

1. A total weight lower than the maximum payload specified in the Kinova arm datasheet (approximately 1 kg);
2. Compact overall dimensions;
3. Proper positioning of the vision system on the upper part of the structure, to minimize the framing of the blades and maximize the visibility of the plant (as the camera was intended to identify different parts of the plant, not to monitor the shears).

The structure was designed using plastic material, opting for a geometry suitable for additive manufacturing, with shapes aimed at minimizing the need for printing supports. After FEM analyses, PLA was selected as the most resistant material among those tested. The geometry was divided into four main components: a lower base to close and seal the original gripper, the main body, housing the electric motor and the Arduino control board, the covers for both the control board and the actuator. Two power solutions were considered for the actuation system: pneumatic and electric. The pneumatic option would have required a system consisting of a silent air compressor with a 6-liter tank, a cylindrical actuator, an end-stroke sensor, a 3/2 solenoid valve. This solution appeared to be too complex, heavy, and bulky.

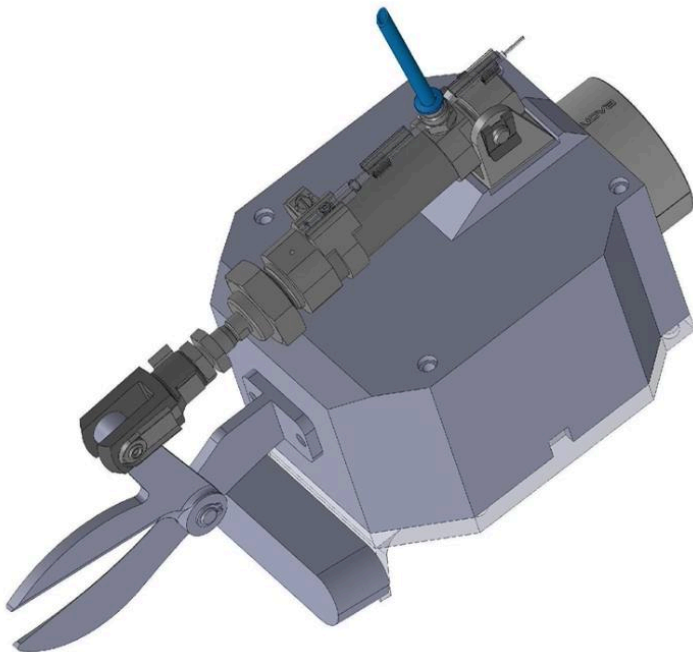


Figure 11 *pneumatic solution, CAD design*

An electric option was ultimately chosen. An automatic electric shear (of unspecified model) available in the lab was disassembled, and the following components were reused: the bypass shears, the reduction gear connected to the electric motor, the actuator, and the main control board. The compact dimensions of the electric actuator (as shown in Figure 14) allowed the construction of a cylindrical housing for the actuator and a dedicated container for the control board.

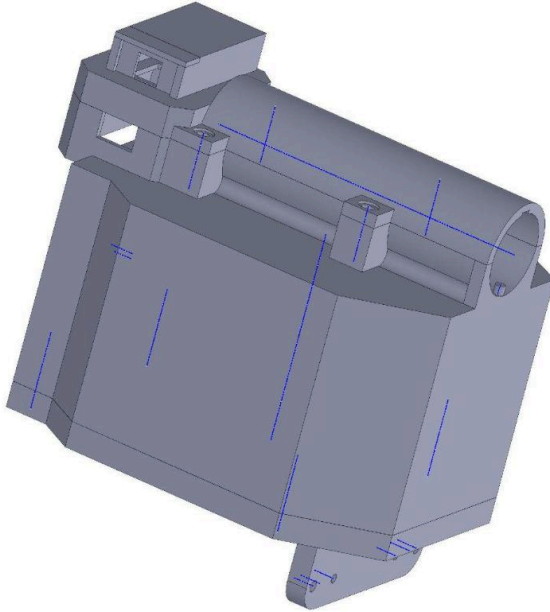


Figure 12 Electric solution showing the glove without the actuators, CAD design (perspective from above)

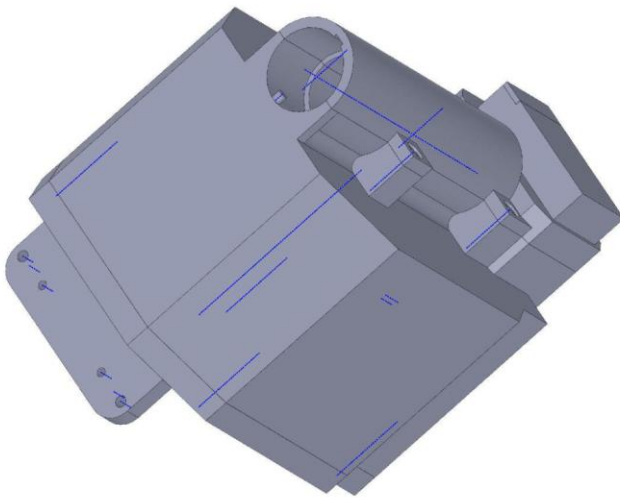


Figure 13 Electric solution showing the glove without the actuators, CAD design (perspective from above)

The bypass shears were mounted on the side of the system, and a dedicated platform for the camera was 3D-printed. An Arduino board was used to control the system, managing the unlocking, cutting, and locking actions. The final system was able to maintain compact dimensions and low manufacturing costs.



Figure 14 Electric solution (lab test, perspective from above)

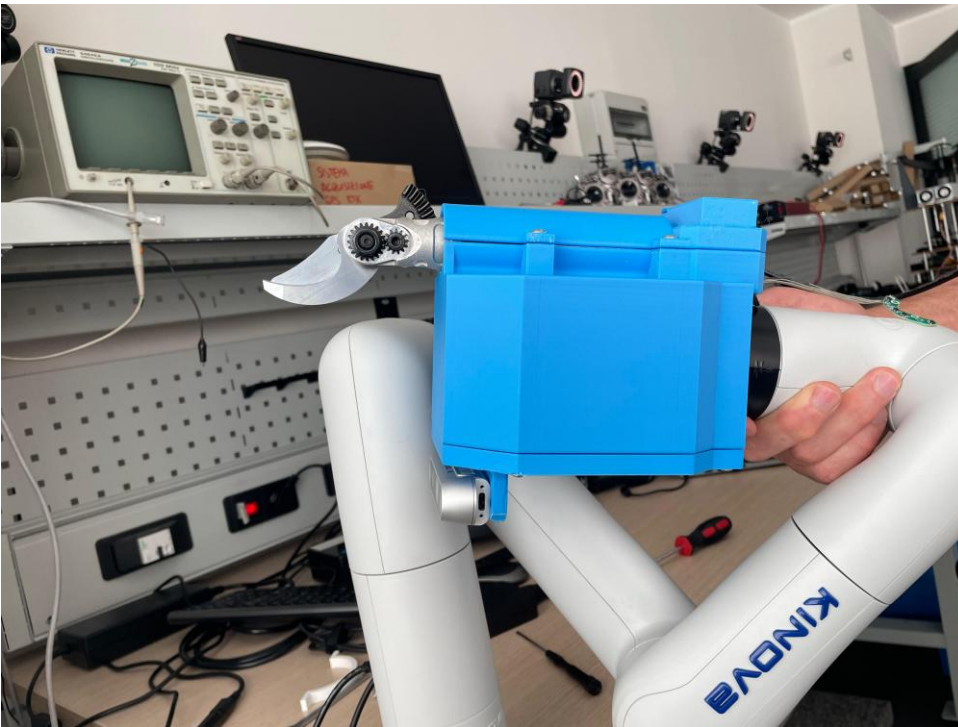


Figure 15 Electric solution (lab test, lateral perspective)

6 Conclusion

In conclusion the overall expectations of the project have been mostly satisfied, at least regarding the different subsections in which the solution has been divided, although the connection between them is not completed.

Here we summarize the overall findings of the project:

- A specific lightweight gripper has been designed for the Kinova Gen3 lite arm. This comprehends the 3D printed holder and the electrical end effector. The design is specifically tailored for the arm and for the application of cutting vines branches.
- The image recognition part has been implemented also specifically for the application of vineyards. The model is able to recognize the different branches starting from the depth image of a plant and to produce a graph corresponding to the skeletonization.
- The pruning policy has been algorithmically translated in such a way that a computer is able to autonomously decide what are the optimal cutting points. This starts from a specific plan graph in input and returns the pixels where the algorithm decides to cut.
- The control pipeline has been set up using fully open source software. It is able to interact with the various hardware components and to manage their synchronization during operation. These softwares also allow for simulation of the system through their visualization interfaces.
- We also considered the possibility that further studies may want to avoid using algorithmic policies by moving towards machine learning. This can be simplified by using virtual reality and automatically generated environments which can be used to train models to recognize plants and also with the help of an expert pruner to collect that to train a cutting point decision unit.

For costs and time motivation the solution is built on top of the hardware that has been furnished for this project. Even though the theoretical principles remain valid, the technical implementation of the solution has to be performed on the very same hardware, otherwise it needs a full revisitation at least of the mechanical and control part.

Even though full operativity of the full system is not achieved, this project represents a step forward towards the application of automatic pruning for vineyards. In fact, as stated in the SOTA section, very few concept of the overall system exists for precision pruning, potentially our project is able to bring new tools to be used in this application (see VR, task constructor, algorithmic policies) and also to validate and integrate existing solutions for specific parts (like image recognition models). Mostly we focused on making the system not only working with an acceptable level of requirements satisfaction, but also easy to use making it as much autonomous as possible, cheap using open source softwares and few hardware resources and flexible trying to implement mostly general solutions not specifically tailored for the hardware or mechanical systems we were using. Some of these constraints were actually very hard to be met, in fact time and cost limitations made it impossible to design a mechanical or control system for a different robot, making impossible an implementation with arms of different sizes, shapes and cost. Full operativity cannot be verified right now since the optimal season to validate the system on the fields is in the next months. Testing it would give the opportunity to implement those small fixes that will properly connect the various parts of the system to make it fully working.

Looking ahead, a number of possible improvements and follow-up activities can be outlined. First, further effort should be invested in integrating the system into a fully operational prototype, ensuring smooth communication and data flow across all modules. This will likely require iterative debugging and

validation cycles, ideally in real vineyard conditions. Second, the adoption of machine learning for pruning policy offers a promising pathway beyond purely algorithmic approaches. By collecting expert-labeled data both in the field and via VR simulation, it will be possible to train models capable of adapting to different vine architectures and evolving pruning practices. Third, there is scope for mechanical optimization: exploring alternative robotic arms or custom manipulators may reduce costs while maintaining or even improving performance. Modular end-effector design would further facilitate adaptability across vineyards of different sizes and grape varieties.

Beyond technical aspects, stakeholder engagement will be crucial to ensure successful adoption. Vineyard owners require evidence of economic return on investment; workers are concerned about employment and must be involved in retraining and system supervision; local communities expect sustainable solutions that preserve the cultural and touristic value of vineyards. Early and continuous collaboration with these groups will help shape both product design and deployment strategies, ensuring that our solution will contribute positively to the ecosystem in which it will operate.

Another important dimension is sustainability. The project has already considered regulatory and environmental requirements, but future work should quantify the robot's life-cycle impacts, particularly regarding energy use, electronic waste, and maintenance logistics. Exploring renewable energy integration, modular recycling of components, and minimal-impact design will be key to aligning with broader sustainability goals in agriculture.

Finally, future research opportunities are abundant. Field trials will provide invaluable data to refine models, validate safety mechanisms, and evaluate pruning accuracy under real-world variability. Partnerships with research institutions may expand the dataset for vine perception and cutting policy, contributing to a global knowledge base for agricultural robotics. Long-term, the technologies developed here such as vision pipelines, pruning policies and robotic control, may be extended beyond vineyards to other perennial crops such as apple orchards, olive groves, or citrus plantations, enhancing the scalability and impact of the solution.

To conclude, although we did not reach yet a fully operational robotic pruning system during the VinPRO project, it still represents a significant advance toward the automation of one of viticulture's most labor-intensive tasks. By combining perception, decision-making, and control into a coherent framework, the project lays the groundwork for future developments that could transform vineyard management. The continuation of this work will be critical to turning this promising prototype into a commercially viable and socially responsible solution.

7 Bibliography

- [1] S. P. Galinato, A. Kendall, and C. A. Miles, “Costs and Profitability for Mechanized Pruning and Harvest in Two Cider Apple Orchard Systems,” *Horttechnology*, vol. 32, no. 3, pp. 275–287, Jun. 2022, doi: 10.21273/HORTTECH04964-21.
- [2] YANMAR Co., “YANMAR Smart Agriculture.” Accessed: Aug. 21, 2025. [Online]. Available: https://www.yanmar.com/global/agri/smart_agri/
- [3] P. di T. PIC4SeR, “PIC4SeR – Interdepartmental Centre on Service Robotics.” Accessed: Aug. 21, 2025. [Online]. Available: <https://pic4ser.polito.it/>
- [4] G. Allegro *et al.*, “Effects of Mechanical Winter Pruning on Vine Performances and Management Costs in a Trebbiano Romagnolo Vineyard: A Five-Year Study,” *Horticulturae* 2023, Vol. 9, Page 21, vol. 9, no. 1, p. 21, Dec. 2022, doi: 10.3390/HORTICULTURAE9010021.
- [5] Landini, “The mechanical pruning of vineyards: when is it convenient?” Accessed: Aug. 12, 2025. [Online]. Available: <https://www.landini.it/as/the-mechanical-pruning-of-vineyards/>
- [6] A. Navone, M. Martini, and M. Chiaberge, “Autonomous Robotic Pruning in Orchards and Vineyards: a Review,” May 2025, Accessed: Aug. 12, 2025. [Online]. Available: <https://arxiv.org/pdf/2505.07318>
- [7] L. Qiang, C. Jianrong, L. Bin, D. Lie, and Z. Yajing, “Identification of fruit and branch in natural scenes for citrus harvesting robot using machine vision and support vector machine,” *International Journal of Agricultural and Biological Engineering*, vol. 7, no. 2, pp. 115–121, 2014, doi: 10.25165/IJABE.V7I2.1215.
- [8] W. Ji, Z. Qian, B. Xu, Y. Tao, D. Zhao, and S. Ding, “Apple tree branch segmentation from images with small gray-level difference for agricultural harvesting robot,” *Optik (Stuttg)*, vol. 127, no. 23, pp. 11173–11182, Dec. 2016, doi: 10.1016/J.IJLEO.2016.09.044.
- [9] N. Shalal, T. Low, C. McCarthy, and N. Hancock, “Orchard mapping and mobile robot localisation using on-board camera and laser scanner data fusion – Part A: Tree detection,” *Comput Electron Agric*, vol. 119, pp. 254–266, Nov. 2015, doi: 10.1016/J.COMPAG.2015.09.025.
- [10] J. Zhang, L. He, M. Karkee, Q. Zhang, X. Zhang, and Z. Gao, “Branch detection for apple trees trained in fruiting wall architecture using depth features and Regions-Convolutional Neural Network (R-CNN),” *Comput Electron Agric*, vol. 155, pp. 386–393, Dec. 2018, doi: 10.1016/J.COMPAG.2018.10.029.
- [11] T. Botterill *et al.*, “A Robot System for Pruning Grape Vines,” *J Field Robot*, vol. 34, no. 6, pp. 1100–1122, Sep. 2017, doi: 10.1002/ROB.21680;WGROU:STRING:PUBLICATION.
- [12] D. Borrenpohl and M. Karkee, “Automated pruning decisions in dormant sweet cherry canopies using instance segmentation,” *Comput Electron Agric*, vol. 207, p. 107716, Apr. 2023, doi: 10.1016/J.COMPAG.2023.107716.
- [13] B. Ma, J. Du, L. Wang, H. Jiang, and M. Zhou, “Automatic branch detection of jujube trees based on 3D reconstruction for dormant pruning using the deep learning-based method,” *Comput Electron Agric*, vol. 190, p. 106484, Nov. 2021, doi: 10.1016/J.COMPAG.2021.106484.
- [14] Y. Majeed, M. Karkee, Q. Zhang, L. Fu, and M. D. Whiting, “A Study on the Detection of Visible Parts of Cordons Using Deep Learning Networks for Automated Green Shoot Thinning in Vineyards,” *IFAC-PapersOnLine*, vol. 52, no. 30, pp. 82–86, Jan. 2019, doi: 10.1016/J.IFACOL.2019.12.501.
- [15] T. Y. Zhang and C. Y. Suen, “A Fast Parallel Algorithm for Thinning Digital Patterns,” 1984.
- [16] X. Li *et al.*, “Efficient three-dimensional reconstruction and skeleton extraction for intelligent pruning of fruit trees,” *Comput Electron Agric*, vol. 227, p. 109554, Dec. 2024, doi: 10.1016/J.COMPAG.2024.109554.
- [17] N. M. Elfiky, S. A. Akbar, J. Sun, J. Park, and A. Kak, “Automation of Dormant Pruning in Specialty Crop Production: An Adaptive Framework for Automatic Reconstruction and Modeling of Apple Trees,” 2015.

- [18] G. Lin, Y. Tang, X. Zou, and C. Wang, "Three-dimensional reconstruction of guava fruits and branches using instance segmentation and geometry analysis," *Comput Electron Agric*, vol. 184, p. 106107, May 2021, doi: 10.1016/J.COMPAG.2021.106107.
- [19] S. A. Akbar, S. Chattopadhyay, N. M. Elfiky, and A. Kak, "A Novel Benchmark RGBD Dataset for Dormant Apple Trees and Its Application to Automatic Pruning," 2016. Accessed: Aug. 12, 2025. [Online]. Available: <https://engineering.purdue.edu/RVL/>
- [20] E. Kok, X. Wang, and C. Chen, "Obscured tree branches segmentation and 3D reconstruction using deep learning and geometrical constraints," *Comput Electron Agric*, vol. 210, p. 107884, Jul. 2023, doi: 10.1016/J.COMPAG.2023.107884.
- [21] Y. Xu, C. Hu, and Y. Xie, "An improved space colonization algorithm with DBSCAN clustering for a single tree skeleton extraction," *Int J Remote Sens*, vol. 43, no. 10, pp. 3692–3713, May 2022, doi: 10.1080/01431161.2022.2102950.
- [22] T. Gentilhomme, M. Villamizar, J. Corre, and J. M. Odobez, "Towards smart pruning: ViNet, a deep-learning approach for grapevine structure estimation," *Comput Electron Agric*, vol. 207, p. 107736, Apr. 2023, doi: 10.1016/J.COMPAG.2023.107736.
- [23] Z. Chen, Y. Wang, S. Tong, C. Chen, and F. Kang, "Grapevine Branch Recognition and Pruning Point Localization Technology Based on Image Processing," *Applied Sciences* 2024, Vol. 14, Page 3327, vol. 14, no. 8, p. 3327, Apr. 2024, doi: 10.3390/APP14083327.
- [24] D. Strnad and Š. Kohek, "Novel discrete differential evolution methods for virtual tree pruning optimization," *Soft comput*, vol. 21, no. 4, pp. 981–993, Feb. 2017, doi: 10.1007/S00500-015-1827-X/FIGURES/11.
- [25] H. Williams *et al.*, "Modelling wine grapevines for autonomous robotic cane pruning," *Biosyst Eng*, vol. 235, pp. 31–49, Nov. 2023, doi: 10.1016/J.BIOSYSTEMSENG.2023.09.006.
- [26] Y. Fu *et al.*, "Skeleton extraction and pruning point identification of jujube tree for dormant pruning using space colonization algorithm," *Front Plant Sci*, vol. 13, p. 1103794, Jan. 2023, doi: 10.3389/FPLS.2022.1103794/BIBTEX.
- [27] S. Kolmanič, D. Strnad, Š. Kohek, B. Benes, P. Hirst, and B. Žalik, "An algorithm for automatic dormant tree pruning," *Appl Soft Comput*, vol. 99, p. 106931, Feb. 2021, doi: 10.1016/J.ASOC.2020.106931.
- [28] W. Villegas Marset, D. S. Pérez, C. A. Díaz, and F. Bromberg, "Towards practical 2D grapevine bud detection with fully convolutional networks," *Comput Electron Agric*, vol. 182, p. 105947, Mar. 2021, doi: 10.1016/J.COMPAG.2020.105947.
- [29] C. A. Díaz, D. S. Pérez, H. Miatello, and F. Bromberg, "Grapevine buds detection and localization in 3D space based on Structure from Motion and 2D image classification," *Comput Ind*, vol. 99, pp. 303–312, Aug. 2018, doi: 10.1016/J.COMPIND.2018.03.033.
- [30] M. Fernandes *et al.*, "Grapevine Winter Pruning Automation: On Potential Pruning Points Detection through 2D Plant Modeling using Grapevine Segmentation," *2021 IEEE 11th Annual International Conference on CYBER Technology in Automation, Control, and Intelligent Systems, CYBER 2021*, pp. 13–18, Jun. 2021, doi: 10.1109/CYBER53097.2021.9588303.
- [31] "16. Fine-tuning and testing of a deep learning algorithm for pruning regions detection in spur-pruned grapevines in: Precision agriculture '21." Accessed: Aug. 14, 2025. [Online]. Available: <https://brill.com/edcollchap/book/9789086869169/BP000020.xml>
- [32] P. Guadagna *et al.*, "Using deep learning for pruning region detection and plant organ segmentation in dormant spur-pruned grapevines," *Precis Agric*, vol. 24, no. 4, pp. 1547–1569, Aug. 2023, doi: 10.1007/S11119-023-10006-Y/FIGURES/4.
- [33] A. Silwal, F. Yandun, A. Nellithimaru, T. Bates, and G. Kantor, "Bumblebee: A Path Towards Fully Autonomous Robotic Vine Pruning," *Field Robotics*, vol. 2, pp. 1661–1696, Dec. 2021, doi: 10.55417/fr.2022051.
- [34] R. Ghianni and F. Tahirovic, "VinPRO GitHub repository."
- [35] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci Robot*, vol. 7, no. 66, May 2022, doi: 10.1126/scirobotics.abm6074.

- [36] D. T. Coleman, I. A. Sucan, S. Chitta, and N. Correll, “Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study,” *JOURNAL OF SOFTWARE ENGINEERING IN ROBOTICS*, vol. 5, pp. 3–16, Jan. 2014.
- [37] M. Gornier, R. Haschke, H. Ritter, and J. Zhang, “MoveIt! Task Constructor for Task-Level Motion Planning,” in *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, May 2019, pp. 190–196. doi: 10.1109/ICRA.2019.8793898.
- [38] S. Macenski, A. Soragna, M. Carroll, and Z. Ge, “Impact of ROS 2 Node Composition in Robotic Systems,” *IEEE Robot Autom Lett*, vol. 8, no. 7, pp. 3996–4003, Jul. 2023, doi: 10.1109/LRA.2023.3279614.
- [39] Kinova robotics, “Discover our Gen3 lite robot.” Accessed: Aug. 13, 2025. [Online]. Available: https://www.kinovarobotics.com/product/gen3-lite-robots#Product_resources
- [40] Arduino, “Arduino nano.” Accessed: Aug. 13, 2025. [Online]. Available: https://store.arduino.cc/products/arduino-nano?srltid=AfmBOoq_pMfm0pu9L9918U-E_CVTIXvx0krwr6_s0N_KjXGat0uWb0uU
- [41] Intel, “Intel® RealSense™ Depth Camera D435i.” Accessed: Aug. 13, 2025. [Online]. Available: Intel® RealSense™ Depth Camera D435i
- [42] Jintao Wu, Guijun Yang, Hao Yang, Yaohui Zhu, Zhenhai Li, Lei Lei, Chunjiang Zhao, Extracting apple tree crown information from remote imagery using deep learning, *Computers and Electronics in Agriculture*, <https://doi.org/10.1016/j.compag.2020.105504>.